# GiraffeGraph senior design report


A Human-in-the-Loop Knowledge Graph Generation Engine for Preventing Hallucination and Schema Drift in LLMs: An Application on Hadith Isnad Chains
Abstract
1 Introduction
- 1.1 Problem
- 1.2 Case study
- 1.3 Objective and approach
- 1.4 Contributions
- 1.5 Scope and limits
- 1.6 Structure

2 Background
- 2.1 Hadith, isnad and matn
- 2.2 Knowledge graphs and schema-constrained extraction
- 2.3 Two failure modes: unsupported facts and schema drift
- 2.4 Grounding and attribution
- 2.5 Human-in-the-loop review and confidence as a routing signal
- 2.6 Derivable facts and decisions

3 Related work
- 3.1 Hadith corpora and labelled resources
- 3.2 Isnad and narrator extraction
- 3.3 Narrator identity resolution
- 3.4 Hadith knowledge graphs and LLM-based structuring
- 3.5 LLM-based knowledge graph construction and its evaluation
- 3.6 Reviews of the field
- 3.7 Positioning

4 Requirements and scope
- 4.1 Users and use cases
- 4.2 Functional requirements
- 4.3 Non-functional requirements
- 4.4 Out of scope

5 System design
- 5.1 Design principles
- 5.2 Architecture
- 5.3 Data model
- 5.4 The pipeline, step by step
- 5.5 Validation checks
- 5.6 Relation identity
- 5.7 The decision record
- 5.8 Plug-ins of the isnad case study
- 5.9 Design decisions and alternatives

6 Implementation
- 6.1 Technology stack
- 6.2 Repository layout
- 6.3 API surface
- 6.4 LLM provider abstraction
- 6.5 Persistence

6.6 Dashboard
6.7 Testing and continuous integration
6.8 How to run it
6.9 Known limitations
7 Evaluation
7.1 Questions
7.2 Data and reference labels
7.3 Protocol
7.4 Conditions
7.5 Extraction results
7.6 What the checks catch
7.7 Model confidence
7.8 Review workload, commits and cost
7.9 A second model: a configuration failure
7.10 Narrator identity
7.11 What we did not measure
7.12 Earlier pilot
8 Discussion
8.1 Where the check belongs
8.2 What the model is needed for
8.3 Confidence
8.4 The cost of a strict commit rule
8.5 Threats to validity
8.6 Limits of the system
9 Conclusion and future work
9.1 Plan for Senior Design Project II
9.2 Beyond hadith
References
Appendix A Reproducing the results
Appendix B Result files
Appendix C Use of AI tools

# A Human-in-the-Loop Knowledge Graph Generation Engine for Preventing Hallucination and Schema Drift in LLMs: An Application on Hadith Isnad Chains

**Senior Design Project Report**

Yahya Efe Kuruçay, Deniz Büyükşahin, Alperen Ulukaya

Supervisor: Dr. Taha Yiğit Alkan

Akdeniz University, Faculty of Engineering, Department of Computer Engineering

CSE 491 Senior Design Project I, Fall 2026–2027

Draft of 12 October 2026

# Abstract

Large language models can turn text into knowledge graph facts, but they add facts the source does not contain and they drift from the schema they were given. This report presents GiraffeGraph Core, an engine in which the model only proposes. Each proposed fact carries a quotation from the source, deterministic checks decide whether a person must look at it, and a fact reaches the graph only with a recorded acceptance, either a reviewer's decision or a named automatic policy. The case study is the isnad, the chain of narrators that precedes a hadith.

We evaluate the engine on 300 hadiths from the six canonical collections, using the narrator labels of an existing platform-derived corpus as reference, with a fixed matching protocol and hadith-level bootstrap intervals. Under loose matching a rule-based parser reaches F1 0.923, a model constrained only by the schema 0.890, and the same model with type descriptions and domain hints 0.956; under strict matching both model conditions are above the parser. A check that the extracted name occurs inside the model's own quotation flags 9% of the schema-only model's proposals. Of the flagged proposals 79% do not match the reference, and they are 84% of all its unmatched proposals; most are names the model transliterated, put in another grammatical case or inferred from a kinship reference. Candidates accepted without review rise from precision 0.914 to 0.985. A check that the quotation occurs in the source finds nothing, because the model quotes correctly and changes the name. The model's self-reported confidence separates matched from unmatched proposals poorly (AUC 0.634). Identifying a narrator by the written name places one chain position in ten under a name shared by several people.

The reference labels are not a human gold standard, no person reviewed the runs, one model produced usable results, and the experiments ran on the engine's in-memory backend. The software has 285 automated backend tests, twelve of which exercise the pipeline on PostgreSQL and Neo4j, and includes a dashboard in which a chain can be read with the source quotation and the acceptance record of every link.



**Keywords:** knowledge graph, large language models, human-in-the-loop, grounding, schema drift, hadith, isnad

# 1 Introduction

## 1.1 Problem

A knowledge graph built from text is useful only if each fact in it can be trusted and traced. Large language models make extraction cheap: given a passage and a schema, a model returns nodes and relations in seconds. The same models make two kinds of mistake that are hard to see afterwards. They return facts that the passage does not support, and they depart from the schema, using a type or a property that nobody defined. Once such output is written to a graph, it looks the same as correct output.

The usual answer is to ask the model for a quotation and a confidence score. A quotation shows where a fact came from, and a score suggests which facts to check. We started from the same idea and found, in our own code, in a pilot and in the experiments of this report, that with the models we tried neither works as assumed. The model quotes the source accurately and then writes a value that is not in the quotation. Its confidence is nearly the same for output that matches the reference and output that does not.

## 1.2 Case study

We study the problem on the isnad, the chain of narrators that introduces a hadith: "A told us, from B, from C". The task suits the question for three reasons. A chain is a graph by nature. Errors matter, because scholars judge a hadith from the people in its chain, and one wrong name changes the judgement. Finally, labelled data exists, so output can be scored instead of inspected.

The task is not unsolved. Fine-tuned models extract narrators and their links accurately when large labelled training sets are available [1], an ontology and a knowledge graph of the six canonical collections have been published [2, 3], and narrator names have been linked to biographical databases [4]. Chapter 3 reviews this work. What we did not find is a study of the step between a general-purpose model and the graph: which checks belong there, what each of them catches, and how a fact's acceptance is recorded.

## 1.3 Objective and approach

The objective is an engine in which a language model proposes graph facts, deterministic checks and human review decide which of them are accepted, and every accepted fact keeps its source quotation and a record of how it was accepted. The second objective is to measure what the checks contribute.

The engine, GiraffeGraph Core, treats the model as an untrusted proposer. A candidate that fails a check is not rejected silently; it goes to a review queue with a reason code. A reviewer's decision is stored with the reviewer's name, the time and the duration, and later processing cannot overturn it. Candidates accepted without a person are recorded as accepted by a named policy. Hadith-specific behaviour is added as plug-ins: a segmentation strategy, a schema template and a rule-based provider.

## 1.4 Contributions

1. **A working engine** with source-grounded candidates, ten reason codes that route a candidate to review and two that record unknown types and properties, relation identity that keeps separate transmissions between the same two narrators apart, an append-only decision record, a chain view, and a test suite that also runs against real PostgreSQL and Neo4j instances (Chapters 5 and 6).

2. **A measurement of the checks inside the engine.** On 300 unseen hadiths, the value-in-quotation check flags 122 of the 145 proposals of a schema-only model that do not match the reference, before any of them is accepted, while a quotation-in-source check flags none (Chapter 7).
3. **A comparison of extraction conditions through one pipeline**: rules, a schema-only model, a model with descriptions and domain hints, and the same model restricted to the chain span, with paired bootstrap intervals. Adding descriptions and hints to a schema-only prompt raises F1 by 0.066; restricting the input to the chain span does not help.
4. **An evaluation kit and data sets**: a scorer with a fixed matching protocol, an analysis of checks, confidence and cost, a 300-hadith test set built from an existing labelled corpus with its provenance, and an annotation sheet with a guideline for a future human-labelled set.
5. **A baseline for narrator identity** that quantifies how often a written name stands for more than one person in the reference labels.

## 1.5 Scope and limits

The report describes a prototype. Narrator identity resolution, rules that need biographical dates, question answering over the graph and authentication are out of scope (Chapter 4). The reference labels come from existing corpora and are not a gold standard produced by independent annotators. No person reviewed the evaluation runs, so the human part of the loop is implemented and tested but not measured. Results come from one model, on the in-memory backend. Chapter 7 lists what was not measured, and Chapter 8 discusses how far the results can be trusted.

## 1.6 Structure

Chapter 2 gives the background on hadith chains and on extraction with language models. Chapter 3 reviews related work. Chapter 4 states requirements and scope. Chapters 5 and 6 describe the design and the implementation. Chapter 7 reports the evaluation, Chapter 8 discusses it, and Chapter 9 concludes with the plan for the second term.

# 2 Background

This chapter introduces the domain of the case study and the concepts that the rest of the thesis depends on.

## 2.1 Hadith, isnad and matn

A hadith is a report with two parts. The isnad (إسناد) is the chain of narrators through whom the report was transmitted, and the matn (متن) is the reported text itself. Some sources and dataset names say sanad for the same chain. From here on we write "chain" for isnad and "report text" for matn.

A chain is written starting from the compiler of a collection and runs back to the Prophet Muhammad or to one of his Companions, the first generation of narrators. Each narrator named in the chain received the report from the narrator named after him. For example, the chain labelled in the Sanadset corpus for hadith 693 of al-Nasai's collection has five narrators: Muhammad ibn Mansur, Sufyan, al-Zuhri, Sa'id and Abu Hurayra [5].

Narrators are joined by transmission formulas (sigha, صيغة). Frequent ones are haddathana ("he narrated to us"), akhbarana ("he informed us"), sami'tu ("I heard") and 'an (عن, "from"). The formula encodes how the report passed from one narrator to the next, and 'an leaves this open. A graph model therefore keeps the formula as a property of the relation between two narrators.

A hadith can reach a compiler through more than one chain. When such parallel chains are written one after another, the letter ح, called tahwil (تحويل), marks the switch to a new chain; we call it the chain-switch marker. In a sample of 300 hadiths that we drew from the six canonical collections, 74 have more than one chain according to the source labels [6]. If the chains of one hadith are not kept apart, the graph gains transmission paths that no chain states.

Narrator identity is the difficult part. The same person can appear as "Sufyan ibn Uyayna", "Ibn Uyayna" or "Sufyan", and different people share the same short form. In the chain above, the text does not say which Sufyan or which Sa'id is meant. Some chains name nobody, as in "from his father, from his grandfather".

In the 27,056 real chains of the AR-Sanad test set, where every name carries a narrator identifier, 815 of 15,301 distinct name strings map to more than one identifier. The name Muhammad maps to 27 narrators and the expression "his father" to 421 [6, 7].

Identity matters because a chain is assessed through its narrators. The biographical literature on narrators, called rijal (رجال), records who each narrator was and how reliable scholars judged that narrator to be; works ordered by generation are one part of it [8]. The datasets we use carry fields from this tradition, such as a generation class and reliability grades attributed to Ibn Hajar and al-Dhahabi [6]. Merging two different narrators into one node can change the verdict on a chain, while a missed merge only leaves a duplicate node. The text alone cannot settle such cases; they need an authority list of narrators and an expert.

The six canonical collections (in Turkish, Kütüb-i Sitte) are those of al-Bukhari, Muslim, Abu Dawud, al-Tirmidhi, al-Nasai and Ibn Majah. A critical review lists the narrow focus of benchmarks on these six books among the weaknesses of the field [9].

## 2.2 Knowledge graphs and schema-constrained extraction

By a knowledge graph we mean a set of typed nodes connected by typed relations, where nodes and relations carry properties; surveys of the concept and of domain-specific construction are [10, 11]. A schema fixes

which node types, relation types and properties are allowed. A schema for narrator chains needs little: a narrator node, a hadith node, and a "narrated from" relation that carries the hadith reference, the chain number, the position in the chain and the transmission formula.

A graph suits this domain because the questions asked about chains are path and network questions. The narrator that two chains have in common is not written in any single passage; it has to be found by traversing both chains.

Large language models (LLMs) can build such graphs from text [12, 13]. Two strategies exist. In schema-first extraction the model receives the allowed types and an output contract, and its output is validated against the schema. In the second strategy the model extracts freely and a later step defines and canonicalises what it produced, as in the Extract, Define, Canonicalize framework [14]. This needs no schema in advance. In our assessment the cost is a second pass, and canonicalisation is one more step in which errors can enter.

## 2.3 Two failure modes: unsupported facts and schema drift

The title of this project names two failure modes. Schema drift means that the output uses types or properties that the schema does not define. The same relation may be called HEARD_FROM in one batch and TOLD or narratedBy in the next. At type level the model writes an undefined node or relation type. At property level it writes an undefined property key or a value outside the allowed list.

Hallucination is the common name for generated content that the source does not support. Ji et al. survey it for natural language generation [15]. That survey does not treat knowledge graph extraction as a task, so its figures do not transfer to our setting. We use the plainer term "unsupported fact".

A single "hallucination rate" is too coarse for extraction, because at least three different things are counted under it:

- **Value not in the source.** The extracted value, for example a narrator name, does not occur in the source text at all.
- **Value not in the model's own quotation.** The model quotes a passage that does occur in the source, but the value it reports does not occur in that passage. This covers real errors and also silent normalisation, for example a name returned in the nominative when the text has it in the accusative. In the second case the model has made a decision that is recorded nowhere.
- **Category error.** The value occurs in the text and the type exists in the schema, but the thing is not of that type, for example the Prophet extracted as a narrator.

Different checks catch the three classes, so Section 7.6 reports them separately. Text2KGBench illustrates why presence in the text is not the same as correctness. Its hallucination measure tests whether the subject or object of a triple can be found in the sentence or the ontology. With Vicuna-13B on the Wikidata-TekGen part of the benchmark, subject hallucination was 0.17 while precision against the reference triples was 0.38 [16].

## 2.4 Grounding and attribution

Grounding ties each output to evidence in the source. Two families of methods are relevant.

Quotation-based checks ask the model to return a verbatim quotation with every candidate fact and then apply string tests: the quotation occurs in the source segment, and the extracted value occurs in the quotation. These tests are deterministic, cost almost nothing and can run on every candidate. They can show that something is wrong; they cannot show that a candidate is right.

Claim-level methods come from the evaluation of free text. FActScore splits a long answer into atomic claims and verifies each one, and SAFE addresses long-form factuality in a similar way [17, 18]. Rashkin et al. define

how to measure whether generated text is attributable to identified sources [19]. SummaC and AlignScore score the consistency of a claim with a text using entailment or alignment models [20, 21]. RARR and Chain-of-Verification use further model calls to revise or verify an output [22, 23]. These methods work at the level of sentences or claims, and the automated ones need at least one model call per item. For a short structured value such as a narrator name, string tests are the cheaper first step. They cannot judge meaning, which is why a human step follows them.

## 2.5 Human-in-the-loop review and confidence as a routing signal

In a human-in-the-loop design a person decides on candidate facts before they are written to the graph. Expert time is the scarce resource, so the system needs a routing signal that selects the candidates a person must see.

The obvious signal is the confidence score that the model reports for its own output, compared with a threshold. Whether such self-reported confidence follows correctness is studied as a calibration problem [24, 25]. The hadith domain advises caution even for a trained classifier: Mahmoud et al. report that 12.9% of the wrong predictions of their narrator identification model were made with a confidence above 0.90 [7]. The essay discussed in Section 2.6 argues that a model's errors look like its correct output on the surface [26].

Two alternatives exist. Deterministic risk flags, such as a failed test from Section 2.4, can route a candidate to review without using the model's opinion of itself. Conformal prediction offers distribution-free error guarantees when a calibration set is available [27]. Whatever the signal, two routing policies are comparable only at the same coverage; Raja compares the error rates of selective serving in this way [28]. Section 7.7 compares the confidence signal with a check at comparable shares of mentions sent to review.

## 2.6 Derivable facts and decisions

The last concept comes from an internal essay of the project, "The Cache and the Ledger" [26]. Its principle is: a fact that can be derived from a source of truth is a cache and should be regenerated, not stored; a fact that cannot be derived is a decision and should be stored once.

The essay's example is a documentation page. "The controller lives in this file" can be recovered from the code. "We considered another design and rejected it" cannot, because the code does not contain the option that was not taken.

The essay supports the principle with counts reproduced by its own scripts. Among 733 Python Enhancement Proposals, 27 of 42 forward "replaces" links were reciprocated by a back-link (64%), although a linter checks the header. It concludes that stored derivable values decay, while append-only decision records whose back-links are derived avoid the problem by construction. It also names limits: decisions have no built-in checksum, and an automated check can flag a problem but cannot certify that a statement is true.

Applied to extraction: a candidate fact that a parser or an LLM extracts from a text is derivable, because it can be extracted again from the same text, although an LLM does not return identical output each time. An acceptance, a rejection, a correction or a merge of two narrators made by a person is not derivable from any source and must be recorded once, with who decided and when. This is why the system keeps a decision trail for every fact.

The principle also explains the second class in Section 2.3. A value that a model silently normalised is neither a plain derivation from the text nor a recorded human decision. A named rule or a person should confirm it before it is accepted.

# 3 Related work

This chapter reviews work on hadith data, on the extraction and identification of narrators, on hadith knowledge graphs, and on the evaluation of knowledge graphs built with large language models (LLMs). For each work we ask who produced the reference, at what level quality was measured, and whether annotator agreement was reported.

The works were found in October 2026 by web search and by reading primary sources. The search was not systematic. We read the full text except in three cases: the thesis of Oumer is not openly accessible, so its figures come from the author's model repository; of Gao et al. we read Sections 1 to 7.1; of Raja, Sections 6 to 8. For the 2023 SemanticHadith paper and the construction frameworks named in Section 3.5 we rely on bibliographic records and report no figures.

## 3.1 Hadith corpora and labelled resources

**Sanadset 650K** has 650,986 records and is described as covering 926 books [5]; the file we downloaded has 956 distinct book values [6]. The text was collected by web scraping, and its components were separated and the narrators tagged "with the help of regular expressions and text matching". The data paper reports no accuracy measurement and no human check. A score against these tags measures agreement with rules. The licence is stated as cc0-1.0 on the dataset card of the Hugging Face mirror [29] and as "CC BY 4.0" on the Mendeley Data record of the original deposit, both read on 12 October 2026.

**The Ifta Sunnah data family** consists of three Kaggle releases of one corpus, which its curator extracted from the Ifta' Sunnah digital library by crawling and parsing [6]. The original corpus has 33 books, 276,347 hadiths and 20,957 narrator profiles. The entities and relations release adds narrator spans and transmission relations, with a fixed split of 223,478 hadiths (156,434 training, 33,521 validation, 33,523 test). The graph release has 257,840 hadiths in which every chain step is linked to a narrator identifier. The labels are the platform's markup, restructured by a third party; the source does not say how the platform produced it and reports no annotator agreement. On 12 October 2026 the Kaggle licence field was stated as "CC BY-SA 4.0" for the original corpus, "Unknown" for the entities release and "CC BY-NC-SA 3.0 IGO" for the graph release; the documentation asks users to follow the terms of the platform.

**AR-Sanad 280K** provides 279,625 artificial chains for training and validation and a test set of 27,056 real chains from the six canonical collections, covering 6,378 narrators, each name with a narrator identifier [7]. According to the paper, a group of 30 experts carried out narrator identification on the source website. The test set contains separated names and no hadith text, so it can test identity but not extraction. The repository has no licence file; the article is listed with a CC BY 4.0 licence URL, and the repository does not say whether this extends to the data [6].

## 3.2 Isnad and narrator extraction

Altammami et al. separate chain from body with n-gram models [30]. The authors labelled 240 hadiths, 40 from each of the six collections, and measured per hadith whether the split point was correct. The bi-gram model was correct on 222 of 240 (92.5%) and the tri-gram model on 48%. Reported error types include parallel chains.

Muther and Smith detect chain spans in long, unpunctuated classical texts [31]. Their corpus has 50 texts, 907,110 tokens and 3,071 chains, annotated by experts in Islamic history who corrected model suggestions. Agreement was measured with 5 annotators on 2,000 lines: Krippendorff's alpha was 0.84 and pairwise

Cohen's kappa ranged from 0.71 to 0.96. Annotators agreed exactly on the start of a chain in 70% of cases and on its end in 53%. A conditional random field model reached F1 0.254 under strict span matching and 0.835 under partial matching. Two points matter for us: the matching criterion changes the score by a large factor, and even experts disagree on where a chain ends.

Oumer fine-tunes the models CAMeLBERT-CA, AraT5 and AraBART on the Ifta entities and relations data [1]. The author's model repository reports, on 33,523 test hadiths, narrator F1 0.98 and relation-pair F1 0.95 for CAMeLBERT-CA; narrator F1 0.97, pair F1 0.95 and F1 0.93 for triples that include the chain number for AraT5; and triple F1 0.48 for AraBART. Pairs and triples are scored by exact match against relations derived from the platform markup. Separation between books in the split is not stated. We found no study that measures LLM-based extraction on this data.

## 3.3 Narrator identity resolution

Mahmoud et al. treat identification as classification of each name mention into one of 18,298 narrators [7]. On artificial validation chains the model reaches micro-F1 92.9 (on a scale of 0 to 100) with a chain error rate of 30.2%, the percentage of chains with at least one wrongly identified narrator. On the real test chains micro-F1 is 83.5, macro-F1 78.8 and the chain error rate 60.6%, ranging by collection from 48.7% (al-Tirmidhi) to 70.5% (al-Nasai). The correct narrator is among the top three candidates in 95.3% of cases and among the top five in 97.1%. In the validation set, 26% of 3,669 instances of common names such as Muhammad and Abdullah were identified correctly. A mention-level score thus hides failure at chain level, and a short ranked list contains the correct narrator more often than the first answer does.

Wirahman links the 185,216 narrator name variants of Sanadset to biographical databases without a labelled reference [4]. Of the variants, 94,628 (51.1%) were linked and 88,343 (47.7%) had no candidate. Accuracy was not measured and thresholds were chosen by inspection. The author leaves 1,000 to 2,000 expert-confirmed links to future work. The two highest-degree nodes of the resulting graph are "his father" (7,612) and "my father" (5,550), which shows the effect of merging unnamed narrators by their surface string.

## 3.4 Hadith knowledge graphs and LLM-based structuring

SemanticHadith is an ontology-driven knowledge graph for the hadith corpus [2]. Its 2026 extension, version 2.0.1, builds a graph from the 34,458 hadiths of the six collections with 303,869 entities, 6,822 persons and 47,496 similarity links [3]. Entities in the text are extracted by a named entity recogniser (micro-F1 97%, macro-F1 85%) and mapped to ontology instances through a name dictionary; unmatched entities are dropped. The ontology has classes for a narrator chain and for its segments. For the graph, three domain experts checked 100 random samples from each collection and no discrepancy was reported; no agreement statistic is given. The work uses no LLM, records no decision per fact and has no check against a source quotation. Persons who share a name are flagged for manual inspection or left out, and a framework that presents ambiguous names to an expert with a candidate list is named as future work.

Rezwan is a corpus of narrations processed with LLMs [32]. Six domain experts scored 1,213 random narrations from 0 to 10 per dimension, without an agreement statistic. Chain and body separation received 9.30 (9.33 in the abstract). The 5.85% of narrations whose error rate exceeded 60% were excluded from the mean.

Çinici builds a biographical dataset with 21 variables for 271 scholars from one volume of a classical biographical work, using retrieval with Gemini 2.5 Flash [33]. The author compared the first 100 records with the original and states that accuracy was high, without a figure. The task is biography, not chains, and there is no graph or approval step.

## 3.5 LLM-based knowledge graph construction and its evaluation

General frameworks such as Extract, Define, Canonicalize, KGGen and iText2KG build graphs from text with LLMs [14, 34, 35].

Text2KGBench is a benchmark for ontology-driven extraction with three metric families: precision, recall and F1 against reference triples; conformance to the ontology; and a hallucination measure that tests whether subject and object occur in the sentence or the ontology [16]. On its Wikidata-TekGen part, Vicuna-13B obtained precision 0.38, recall 0.34 and F1 0.35, ontology conformance 0.83 and subject hallucination 0.17.

Gao et al. estimate the accuracy of a whole graph from a sample [36]. On their MOVIE graph, a random sample of 174 triples from 174 entities took 3.53 hours, while two-stage cluster sampling of 178 triples from 24 entities took 1.4 hours, with a margin of error of about 5% in both. The estimate is unbiased and comes with a confidence interval.

## 3.6 Reviews of the field

Haque and Islam assess 32 records on six dimensions: corpus realism, evaluation design, transfer, reproducibility, expert involvement and scholarly use [9]. Among the 8 rows of their assessment table, expert involvement is rated full only for Rezwan. They find a benchmark culture limited to the six collections, a gap between synthetic and real data, metrics that cannot be compared, and sparse expert validation.

Mhamedi et al. review 57 studies [37]. In their table of narrator extraction, reported F1 values range from 33% to 99.63%, test sets have 102 to 200 hadiths, and accuracy, precision and F1 are mixed.

## 3.7 Positioning

| Work | What it provides | How quality was measured | What is missing for our purpose |
|---|---|---|---|
| Sanadset 650K [5] | 650,986 records with tagged narrators | Not measured; tags come from regular expressions | A checked reference; narrator identity |
| Ifta data family [6] | Narrator spans, relations and identifiers | No agreement figure; origin of markup not described | Licence confirmation; an LLM-based study |
| AR-Sanad 280K [7] | 27,056 real chains with identifiers; classifier | Micro and macro F1, chain error rate, top-k | Hadith text, so no extraction test |
| Altammami et al. [30] | Chain and body segmentation | Per-hadith accuracy on 240 hadiths | Narrator-level output |
| Muther and Smith [31] | Expert-annotated corpus; span detection | Strict and partial span F1; agreement of 5 annotators | Narrator names and relations |
| Oumer [1] | Fine-tuned narrator and relation extraction | Exact match on 33,523 test hadiths (author's model repository) | A setting without training data; decision records |
| Wirahman [4] | Automatic linking of 185,216 name variants | Not measured | Labelled reference; human confirmation |
| SemanticHadith 2.0.1 [3] | Ontology and graph of 34,458 hadiths | Recogniser F1; expert check of samples | Decision per fact; source quotations |
| Rezwan [32] | LLM-processed corpus | Expert scores from 0 to 10 | Count-based precision and recall |
| Çinici [33] | LLM-built biographical records | One author, 100 records, no figure | Measured error classes; approval step |
| Text2KGBench [16] | Benchmark and three metric families | Reference triples, ontology conformance, text presence | Value-in-quotation test; human step |

| Work | What it provides | How quality was measured | What is missing for our purpose |
|---|---|---|---|
| Gao et al. [36] | Sampling design for graph accuracy | Unbiased estimate with confidence interval | Not applied here; a candidate for estimating accuracy beyond the labelled sample |

Much of what such a project might claim already exists. A graph of the six collections has been published, and its ontology models chain segments as classes. Narrator linking has labelled real data and a measured baseline. Fine-tuned models extract narrators and transmission relations with F1 values of 0.95 and above against platform labels, according to the author's model repository. An expert-annotated corpus with agreement figures exists for chain spans, and a test for presence in the text is an established metric. We therefore do not claim a new schema, a new representation of the transmission event, or higher extraction accuracy.

Three things we did not find in the works we reviewed: a study that measures the separate contribution of each validation step between an LLM and a graph; a check that the extracted value occurs in the model's own quotation, reported as its own error class; and a record of who or what accepted each fact. These are the subject of this thesis: the system places deterministic checks and a human review queue between the model and the graph, counts what each step catches, and stores an acceptance record for every fact. Because the search was not systematic, "not found" does not mean "does not exist".

# 4 Requirements and scope

This chapter states what the system must do, where each requirement is implemented and which test checks it. Implementation paths are relative to `backend/src/app/` unless they start with another directory. Test files are in `backend/tests/unit/` or `backend/tests/integration/`. We list a requirement only if its code is on the `feat/hadith-prototype` branch.

## 4.1 Users and use cases

The system has no user accounts, so the roles below are not enforced by the software.

- **Schema owner.** Creates a project, defines node and relation types or applies a template schema, and activates a schema version.
- **Operator.** Loads source text, starts processing and commits a run to the graph.
- **Reviewer.** A person with domain knowledge who accepts, rejects or edits the candidates in the review queue. The reviewer types a name, which is stored with every decision.
- **Reader.** Browses the committed graph and, in the case study, reads a hadith with its narrator chains, quotations and acceptance records.

The use cases are: set up a project with a schema, load texts, process a text into validated candidates, decide the open review items, commit a run, and trace a committed fact back to its source text and its acceptance.

## 4.2 Functional requirements

| ID | Requirement | Implemented in | Verified by |
|---|---|---|---|
| FR1 | A project has versioned schemas with node types, relation types, properties and merge keys. Names are ASCII identifiers. Only a draft version can be edited. | `application/use_cases/schema_management.py`, `domain/schema/identifiers.py` | `test_project_schema_api.py` |
| FR2 | Creating a project can apply a template schema and activate it in the same request. | `ProjectSetupService`, `infrastructure/template/catalog.py` | `test_template_project_api.py` |
| FR3 | Source text is loaded one unit at a time or in bulk. A repeated external | `IngestionManagementService` | `test_ingestion_api.py`, `test_bulk_ingest_api.py` |

| ID | Requirement | Implemented in | Verified by |
|---|---|---|---|
| | reference is reported per item. | | |
| FR4 | A source unit is cut into segments by a named strategy: `paragraphs`, or `isnad` with one segment per narrator chain and source offsets. | `domain/extraction/segmentation.py`, `domain/hadith/segmentation.py` | `test_hadith_segmenter.py`, `test_isnad_segmentation.py` |
| FR5 | A provider proposes candidates per segment. A failed call does not discard the other calls. An unknown type or undeclared property is dropped alone and recorded as a schema proposal. | `ExtractionManagementService.extract_candidates` | `test_isnad_pipeline_api.py`, `test_grounding_extraction_api.py` |
| FR6 | Candidates are validated against the schema: type, required properties, merge keys, endpoint types, allowed values, confidence thresholds and duplicates. | `PipelineManagementService.validate_run` | `test_pipeline_api.py` |
| FR7 | The quotation of an extracted candidate must occur in its segment, and a node's merge-key value must occur in the quotation. A failed check sends the candidate to review. | `_evidence_findings`, `_merge_key_grounding_findings`, `domain/hadith/grounding.py` | `test_hadith_grounding.py`, `test_grounding_extraction_api.py` |
| FR8 | Two relations between the same two nodes with different | `domain/shared/resolution.py`, `infrastructure/persistence/neo4j/repositories.py` | `test_resolution.py`, `test_commit_api.py` |

| ID | Requirement | Implemented in | Verified by |
|---|---|---|---|
| | merge-key values are stored as two edges. | | |
| FR9 | A reviewer accepts, rejects or edits a review item. Each decision is stored with reviewer, time and optional duration; decisions are only appended. Automatic steps do not change a decided candidate. | `ReviewManagementService.decide_review_item`, `validate_run`, table `review_decisions` | `test_review_decisions_api.py`, `test_review_api.py` |
| FR10 | For every validated candidate of a run, the system reports the human decision or the automatic policy that accepted it. | `ReviewManagementService.list_acceptance_records` | `test_review_decisions_api.py` |
| FR11 | A run is committed once, and only when no candidate is pending and no review item is open. | `CommitManagementService.commit_run` | `test_commit_api.py` |
| FR12 | Extraction, validation and commit run as queued jobs consumed by a worker, or in one synchronous request for a list of source units. | `processing_job_management.py`, `interfaces/cli/processing_jobs_worker.py`, `SourceUnitProcessingService` | `test_processing_jobs_api.py`, `test_bulk_ingest_api.py` |
| FR13 | The committed graph can be read, and a committed hadith can be read as ordered chains with quotations and | `application/use_cases/exploration_management.py` | `test_exploration_api.py`, `test_chains_api.py` |

| ID | Requirement | Implemented in | Verified by |
|---|---|---|---|
| | acceptance records. | | |
| FR14 | A browser dashboard drives the steps above. | `frontend/src/modules/projects/` | No automated test; CI builds it |

## 4.3 Non-functional requirements

| ID | Requirement | Implemented in | Verified by |
|---|---|---|---|
| NFR1 | Traceability: a committed node or edge links to its source unit and supporting segments. | `DERIVED_FROM`, `SUPPORTED_BY` and `RelationEvidence` written by `Neo4jCanonicalGraphRepository` | `test_exploration_api.py` (in-memory), `test_realdb_isnad_flow.py` (`real_db`) |
| NFR2 | Atomic graph write: a failed commit leaves no part of the run in the graph. | `commit_bundle` runs `_write_bundle` in one `execute_write` transaction | `test_realdb_isnad_flow.py` (`real_db`) |
| NFR3 | The engine runs and is tested without a database, a network or an API key. | `infrastructure/persistence/memory/`, `infrastructure/llm/rule_based_isnad_provider.py` | The default test run, `test_isnad_baseline_script.py` |
| NFR4 | Domain behaviour is added as a segmentation strategy, a template or a provider. | `SegmentationStrategy`, `StaticDomainTemplateCatalog`, `StaticLLMProviderRegistry`, `bootstrap/container.py` | `test_isnad_pipeline_api.py`, `test_rule_based_isnad_provider.py` |
| NFR5 | Repeated model calls are served from a cache, and token usage is reported per call. | `infrastructure/llm/cached_provider.py`, `infrastructure/llm/anthropic_provider.py` | `test_llm_cache_provider.py`, `test_anthropic_provider.py` |
| NFR6 | The database schema is created only through migrations and matches the ORM models. | `backend/migrations/versions/` `0001` to `0006` | `test_review_decisions_migration.py`, `test_realdb_migrations.py` (`real_db`) |
| NFR7 | Backend code passes lint and strict type checking on every push. | `backend/pyproject.toml`, `.github/workflows/ci.yml` | The CI workflow |

Tests marked `real_db` need running databases and are skipped otherwise (Section 6.7). NFR4 holds with the exceptions listed in Section 5.1.

## 4.4 Out of scope

We checked each item against the code.

- **Narrator identity resolution.** The `Narrator` type has the single merge key `name`, compared after Unicode NFKC normalisation, whitespace collapsing and case folding only (`normalize_text`). People with the same written name share a node, and one person written in two ways becomes two nodes.
- **Domain rules that need biographical dates.** The isnad schema has no date properties, and validation has no hook for rules such as checking that two narrators were contemporaries.
- **Authenticity grading.** No type or property holds a grade for a hadith or a narrator.
- **Authentication and users.** No router has an authentication dependency. The reviewer name is free text.
- **Undo of a committed run.** The API has 33 `GET` and 20 `POST` endpoints and no `PUT`, `PATCH` or `DELETE`. A completed run cannot be changed.
- **Turkish hadith text.** The normalisation and the lexicon of transmission formulas cover Arabic script only. Only the dashboard interface is in Turkish.
- **Question answering over the graph.** The read endpoints list and show nodes, relations and chains. An `EmbeddingProvider` protocol is declared in `application/ports/llm_provider.py` without an implementation, and there is no retrieval endpoint.

Two schema fields are stored but not enforced. The `data_type` of a property is sent to the model and is not checked during validation. The `directional` flag of a relation type is not used after it is stored.

# 5 System design

This chapter describes the design as implemented on the `feat/hadith-prototype` branch. Paths are relative to `backend/src/app/`. Several decisions answer defects found in a review of the earlier code (design analysis, 30 September 2026); we name the defect where it explains the design.

## 5.1 Design principles

**The model proposes and cannot write to the graph.** A provider returns suggested nodes and relations, stored as `CandidateNode` and `CandidateRelation` rows with status `pending`. The only graph write, `CanonicalGraphRepository.commit_bundle`, is called only from `CommitManagementService.commit_run` and receives only `validated` candidates.

**Every fact carries a quotation.** A candidate holds `EvidenceRef` values: a segment id, a quotation and optional offsets. An extracted candidate without evidence is flagged `missing_evidence`. This is a check, not a storage constraint: a reviewer can still accept such a candidate, and candidates entered by hand are not checked.

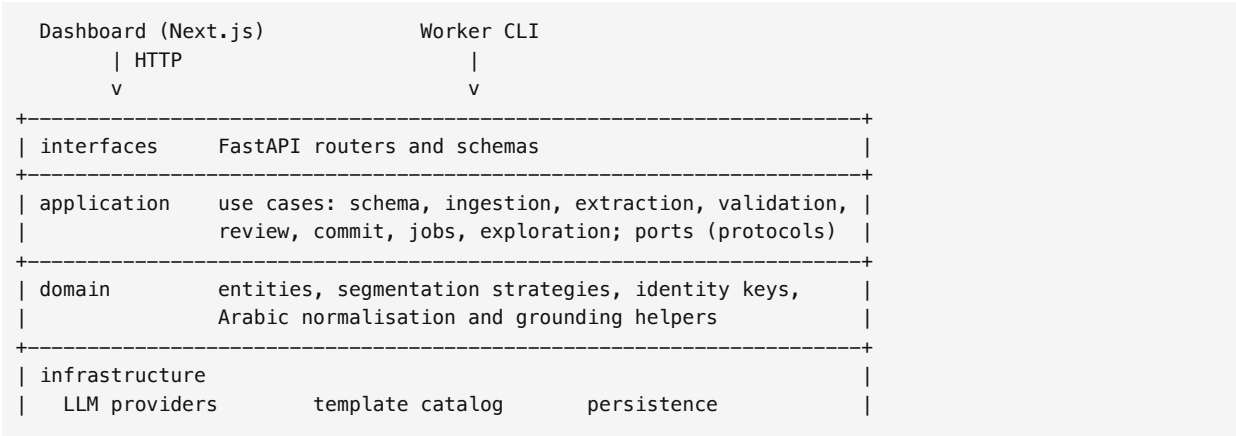
**Checks flag for review instead of silently rejecting.** A failed grounding, value, threshold or duplicate check sets status `review` and creates a review item with a reason code. Structural errors that leave a candidate unusable set `rejected`, but each is stored as a validation result.

**Every acceptance is recorded as a human decision or a named policy.** A validated candidate is traced to a row in `review_decisions` or to the policy `auto_validated` with the rule that passed it.

**Domain behaviour is plugged in; engine code stays generic.** Segmentation is a `SegmentationStrategy`, a schema comes from a template, and extraction goes through an `LLMProvider`. The use cases for extraction, validation, review and commit contain no narrator or hadith names. There are three exceptions. Validation imports its text comparison from `domain/hadith/grounding.py`, so the Arabic folding applies to every project. The chain read model (`domain/exploration/chains.py`, `ChainExplorationService`, the `/chains` endpoints) names the `Narrator` and `NARRATED_FROM` types. The plug-ins are registered in code in `bootstrap/container.py`.

## 5.2 Architecture

The backend is one FastAPI application in four layers. The application and domain layers import nothing from the infrastructure or interface layers: use cases depend on the protocols in `application/ports/`, and the infrastructure layer implements them.



```
|   noop, openai,        blank, story,          in-memory, or          |
|   anthropic,           isnad                  Postgres + Neo4j       |
|   rule_based_isnad                                                   |
+----------------------------------------------------------------------+
        |                                       |
        v                                       v
  Postgres: schemas, source text, runs,    Neo4j: committed nodes and
  candidates, validation results,          edges, links to source units
  review items, decisions, deltas          and segments, graph deltas
```

Postgres holds configuration, source text, everything not yet accepted, and the audit trail. Neo4j holds only what has been committed, with links back to source units and segments. The repository's ADR-002 gives the reason: metadata is structured records, while the committed graph is highly connected and read by traversal. With this split, a candidate never exists in the graph store. The cost is that the two stores are not updated in one transaction (Section 6.9).

An in-memory backend implements the same repository protocols with dictionaries. It is the default when `REPOSITORY_BACKEND` is not `postgres`, and the test suite uses it.

## 5.3 Data model

| Entity | Main fields | Stored in |
|---|---|---|
| Project | name, `template_id`, status | `projects` |
| Schema version | `version_no`, status (`draft`, `published`, `active`, `retired`) | `schema_versions` |
| Node type | name, `display_name`, `merge_keys`, `confidence_threshold` (default 0.5), `extraction_hints` | `node_types` |
| Relation type | name, `from_type`, `to_type`, `merge_keys`, `confidence_threshold` (default 0.7) | `relation_types` |
| Property | name, `data_type`, required, `enum_values` | `property_definitions` |
| Source unit | `raw_content`, `external_ref`, status | `source_units` |
| Segment | order, text, metadata (strategy, chain number, source offsets) | `segments` |
| Run | source unit, schema version, provider, model, status | `processing_runs` |
| Candidate node, candidate relation | type, properties, confidence, evidence refs, status; a node has a `merge_key_payload`, a relation two candidate node refs | `candidate_nodes`, `candidate_relations` |
| Validation result | `candidate_ref`, `rule_code`, severity, result, message | `validation_results` |
| Review item | `item_type`, `item_ref`, `reason_code`, payload, status | `review_items` |
| Review decision | see Section 5.7 | `review_decisions` |
| Graph delta | counts of created, merged and rejected items of one run | `graph_deltas`, `GraphDelta` node |
| Canonical node | labels `CanonicalNode` and the type name; key `project_id`, `node_type`, `merge_key` | Neo4j |
| Canonical edge | relationship named after the relation type; key `project_id`, `relation_type`, | Neo4j |

| Entity | Main fields | Stored in |
|---|---|---|
| | `relation_identity_key` | |

Evidence refs are stored as JSON in the candidate rows. Only a draft schema version can be edited, so the types of a run do not change.

## 5.4 The pipeline, step by step

We follow one hadith with the reference `Muslim:978`, whose text contains the chain-switch marker ح. The segmenter returns two chains for it. The reference labels give three: the third starts after the report text has begun, and the segmenter does not return it.

1. **Ingest.** The text becomes a source unit with `external_ref = "Muslim:978"`, bound to the project's active schema version.
2. **Start a run.** A `ProcessingRun` records the schema version, provider and model. The source unit is marked `processing`, and no second run can start in that state.
3. **Segment.** The `isnad[ strategy returns one segment per chain it finds; it starts a new chain only at the marker ح, so most hadiths give one segment. Each segment text is the exact substring ] {lang=ar dir=rtl}raw_content[start:end]`, and the offsets are stored in the segment metadata. If no chain is found, the whole text becomes one segment marked `fallback: whole_text`.
4. **Extract.** With the `isnad` strategy the engine makes one provider call per segment; with `paragraphs` it sends the segments of a source unit in one call. The payload holds the source unit reference, the schema with its hints, and the segment. The answer is parsed with `StructuredExtractionContract`. A node or relation with an unknown type is dropped and recorded as a schema proposal; an undeclared property is removed the same way. An answer that cites a segment outside the call, uses an unknown node ref, or connects nodes of the wrong types fails that call. Nodes from different calls with the same type and merge-key value become one candidate that keeps all quotations. The run fails only if every call fails.
5. **Validate.** `validate_run` applies the checks of Section 5.5, sets each candidate to `validated`, `review` or `rejected`, and builds the review items.
6. **Review.** A reviewer decides the open items (Section 5.7). After each decision, relations that waited on a node are refreshed.
7. **Commit.** `commit_run` refuses a run that already has a graph delta, a pending candidate, an open review item, a validated relation whose endpoint is not validated, a validated relation without a merge-key value, or two validated relations that would become one edge with different properties. Otherwise the validated candidates are written in one graph transaction, the delta is stored, and the run becomes `completed`.
8. **Read.** The chain endpoints return the hadith with its chains in order, each link with its quotation and acceptance record.

Steps 2 to 5, and step 7 for runs without open items, can run in one request (`POST /api/projects/{project_id}/source-units/process`). Steps 4, 5 and 7 can also run as queued jobs.

## 5.5 Validation checks

| Reason code | Trigger | Effect | Rationale |
|---|---|---|---|
| `node_type_exists`, `relation_type_exists` | Type is not in the schema version | Rejected | Graph labels come only from the schema; reached only by manual entries |

| Reason code | Trigger | Effect | Rationale |
|---|---|---|---|
| `required_node_property`, `required_relation_property`, `node_merge_key` | A required property or a node merge-key value is missing | Rejected | Incomplete by the schema; a node without identity cannot be merged |
| `relation_candidate_refs`, `relation_from_type`, `relation_to_type` | An endpoint is not in the run, or its node type differs from the relation type | Rejected | Schema conformance |
| `relation_dependency_rejected` | An endpoint node is rejected | Rejected | An edge needs both nodes |
| `missing_evidence` | Extracted candidate has no evidence ref | Review | A fact without a quotation cannot be checked |
| `quote_not_in_segment` | Quotation is empty or not found in the cited segment | Review | The quotation may be invented |
| `value_not_in_quote` | A string merge-key value of a node occurs in none of its quotations and differs from the source unit's `external_ref` | Review | The name may be altered or invented |
| `property_value_not_allowed` | A string value is outside the property's `enum_values` | Review | Value drift |
| `node_confidence_threshold`, `relation_confidence_threshold` | Confidence is below the type's threshold | Review | The model reports doubt |
| `missing_relation_merge_key` | A relation lacks a value for a declared merge key | Review | A reviewer can supply it |
| `relation_dependency_review` | An endpoint node is in review | Review | The edge waits for its node |
| `potential_duplicate_node`, `potential_duplicate_relation` | Another candidate of the run has the same identity | Review for all but one | They would become one graph element |
| `schema_type_proposal`, `schema_property_proposal` | Extraction met an unknown type or an undeclared property | Candidate dropped or property removed; review item of type `schema_proposal` | Drift is recorded, the call survives |
| `candidate_node_valid`, `candidate_relation_valid` | All checks passed | Validated | Basis of `auto_validated` |
| `canonical_node_match`, `canonical_relation_match` | The same identity is already in the graph | No status change | Context for the reviewer |
| `human_review_decision` | The candidate has a recorded decision | Status follows the decision | Section 5.7 |

Quotations and values are compared after `normalize_for_grounding`. It applies the engine's `normalize_text` (Unicode NFKC, collapsed whitespace, case folding) and then `normalize_arabic`, which removes short-vowel and Quranic marks and the elongation character, replaces punctuation with spaces, and folds the alef forms to ى ,ا to ة ,ي to ؤ ,ه to و and ئ to ي. The comparison is substring containment, and an empty quotation never matches. Relation property values are not compared with the quotation.

A failed check always routes to review, whatever the confidence. Confidence is a number the model reports about itself, so a fabricated fact that fits the schema can carry a high value; the check result does not depend on it. The candidate is not rejected either, because a failure can be legitimate: a model that adds a definite article to a name fails `value_not_in_quote`, and a person should decide that case. The dependency refresh closes only items with a dependency reason, so it leaves these open.

The evidence and value checks run only when the run has a successful extraction execution; candidates entered through the API are treated as written by the operator.

## 5.6 Relation identity

In the earlier code an edge was merged on its two endpoints and its type, and its properties were then overwritten. Two transmissions between the same two narrators in two hadiths became one edge, and the second replaced the hadith reference and wording of the first. Duplicate detection among candidates used all properties as the key, so it saw two different candidates and raised no review item. The loss was silent.

A relation type now declares `merge_keys`, which must name properties declared on the type. `build_relation_identity_key` hashes the normalised merge-key values with SHA-256, and the Neo4j statement merges on that key:

```
MERGE (from_node)-[r:`NARRATED_FROM` {project_id: $project_id,
       relation_type: $relation_type,
       relation_identity_key: $relation_identity_key}]->(to_node)
```

Candidate-level detection uses the same identity. `build_candidate_relation_resolution_key` joins the relation type, the identity keys of both endpoints and the relation identity key, and ignores properties outside the merge keys. Two candidates are therefore duplicates exactly when they would become the same edge.

A type without merge keys keeps one edge per node pair, but a second candidate for that pair now goes to review, and the commit refuses two validated relations that resolve to one edge with different properties.

## 5.7 The decision record

Decisions are stored in `review_decisions`. The repository protocol offers only `add_decision` and `list_decisions_by_run`. Re-validation replaces a run's review items but not its decisions.

A decision stores the review item, run, item type and item reference, the action (`accept`, `reject`, `edit`), `reviewer_id`, an optional note, an optional `duration_ms`, the time, and for an edit the candidate before and after. An edit is applied to a copy of the candidate and checked against the schema and against duplicates in the run; a refused decision stores nothing. A second decision on the same item is appended, and the latest one counts. A completed run accepts no decisions.

Human decisions are final for the automatic steps. `validate_run` reads the decisions first: a decided candidate takes its status from the latest decision, receives the result `human_review_decision`, skips the checks and keeps its review item. Duplicate handling never moves a decided candidate back to review, and the dependency refresh skips decided relations.

Acceptance records are computed on request by `list_acceptance_records` and are not stored. For each validated candidate the record holds the latest accepting decision, or the policy `auto_validated` with the rule code that passed the candidate.

Two limits apply: the reviewer name is not authenticated, and a decision refers to a candidate id of one run, so a new run over the same text does not reuse it.

## 5.8 Plug-ins of the isnad case study

**Segmentation strategy.** `IsnadSegmentation` wraps the rule-based `segment_isnad[. It normalises the text while keeping a map to source offsets, splits at the marker ح, finds transmission formulas from a lexicon of 26 entries, and ends the chain at the first cue of the report text. The strategy sets ] {lang=ar dir=rtl}one_call_per_segment.`

**Schema template.** The `isnad` template declares `Narrator` (merge key `name`), `Hadith` (merge key `hadith_ref`), `NARRATED_FROM` from narrator to narrator (merge keys `hadith_ref`, `chain_no`, `position`; property `sigha` for the wording) and `TRANSMITS` from narrator to hadith. It also names the segmentation strategy. Project creation applies it when `apply_template_schema` is set.

**Rule-based provider.** `rule_based_isnad` is the condition without a language model. It ignores the prompt, runs the same segmenter on each segment, and returns narrator nodes with the name as written and one `NARRATED_FROM` relation per consecutive pair. It creates no `Hadith` nodes. Its quotations are slices of the segment, so its output passes the grounding checks by construction.

**Extraction hints.** Each type carries a list of hint strings that travel to the model inside the schema part of the payload, for example "Copy the narrator name verbatim from the segment text."

## 5.9 Design decisions and alternatives

| Decision | Alternative considered | Reason |
|---|---|---|
| Edge identity from relation merge keys | A node for each transmission event | Smaller change to the generic model |
| A failed check routes to review | Reject it, or accept above a confidence threshold | Rejection hides legitimate variants; the threshold trusts a self-reported number |
| An unknown type drops one candidate and is recorded | Fail the whole answer, as before | Valid candidates are kept and drift can be counted |
| Append-only decisions with computed acceptance records | An event log from which the graph is rebuilt | Enough for an audit trail; rebuild and undo are not implemented |
| A rule-based segmenter available before the model, with one call per chain segment | One call per hadith, with the model finding the chains | Exact substrings and offsets, isolated failures, a baseline without a model. The evaluation found no gain in extraction quality from it (Section 7.5), so both strategies remain selectable |
| One graph transaction per run | One transaction per node, as before | A failed commit leaves no partial graph |

We accept three limits knowingly. A narrator node is identified by the exact written name, diacritics included, so the graph merges people who share a name and splits one person written in two ways. Chains that start after the boundary of the report text are not segmented, because transmission particles there mostly belong to the narrative. The grounding checks apply only to candidates that come from an extraction call.

# 6 Implementation

This chapter describes the code on the `feat/hadith-prototype` branch of the `giraffegraph-core` repository. Most of the code on this branch was written with AI coding assistants under the authors' direction and is verified by the automated test suite described in Section 6.7.

## 6.1 Technology stack

| Part | Technology | Declared version |
|---|---|---|
| Backend | Python, FastAPI, Uvicorn, Pydantic | Python `>=3.12` (3.12 in the Docker image and in CI), `fastapi>=0.115`, `uvicorn>=0.34`, `pydantic>=2.10` |
| Metadata store | PostgreSQL, SQLAlchemy, psycopg, Alembic | image `postgres:16-alpine`, `sqlalchemy>=2.0`, `psycopg>=3.2`, `alembic>=1.15` |
| Graph store | Neo4j and its Python driver | image `neo4j:5`, `neo4j>=6.0,<7.0` |
| Model access | `anthropic` SDK; OpenAI-compatible HTTP API through the standard library | `anthropic>=1.13,<2.0` |
| Dashboard | Next.js, React, TypeScript | `next ^15`, `react ^19`, `typescript ^5`; Node 22 in CI |
| Tooling | uv, pytest, ruff, mypy in strict mode | `pytest>=8.3`, `ruff>=0.11`, `mypy>=1.15` |

Versions are the constraints declared in `backend/pyproject.toml`, `frontend/package.json` and `docker-compose.yml`. Docker Compose starts five services: `postgres`, `neo4j`, `backend`, `worker` and `dashboard`.

## 6.2 Repository layout

```
backend/
  src/app/domain/          entities, segmentation, hadith helpers, identity keys
  src/app/application/     use cases and ports
  src/app/infrastructure/  LLM providers, template catalog, persistence
  src/app/interfaces/      API routers and worker CLI
  src/app/bootstrap/       application wiring
  migrations/versions/     Alembic migrations 0001 to 0006
  scripts/                 baseline and experiment runners
  tests/unit, tests/integration
frontend/src/modules/projects/   dashboard components, API client, types
evaluation/                scorer, analysis, annotation kit, their tests
```

## 6.3 API surface

The 13 routers in `interfaces/api/routers/` define 53 endpoints: 33 `GET` and 20 `POST`.

| Group | Count | Representative endpoints |
|---|---|---|
| Projects, templates | 4 | `POST /api/projects`, `GET /api/templates` |
| Schema | 7 | `POST /api/schema-versions/{schema_version_id}/node-types` |
| Pipeline definitions | 3 | `GET /api/schema-versions/{schema_version_id}/pipeline-` |

| Group | Count | Representative endpoints |
|---|---|---|
| | | definitions |
| Ingestion and runs | 8 | POST /api/projects/{project_id}/source-units/bulk, POST /api/projects/{project_id}/source-units/process, POST /api/source-units/{source_unit_id}/runs |
| Segments, candidates, validation | 10 | POST /api/source-units/{source_unit_id}/segments/generate, POST /api/runs/{run_id}/validate, GET /api/runs/{run_id}/review-items |
| Extraction | 2 | POST /api/runs/{run_id}/extract-candidates |
| Review | 4 | GET /api/projects/{project_id}/review-queue, POST /api/review-items/{review_item_id}/decision, GET /api/runs/{run_id}/acceptance-records |
| Commit | 2 | POST /api/runs/{run_id}/commit, GET /api/runs/{run_id}/graph-delta |
| Jobs | 4 | POST /api/runs/{run_id}/processing-jobs, POST /api/processing-jobs/process-next |
| Graph | 5 | GET /api/projects/{project_id}/graph/nodes |
| Chains | 3 | GET /api/projects/{project_id}/chains/hadith, GET /api/projects/{project_id}/chains/narrator |
| Health | 1 | GET /health |

Application errors map to the status codes 404, 409, 422 and 502 in `error_handlers.py`.

## 6.4 LLM provider abstraction

The port `LLMProvider` has two methods. `generate_structured` returns the answer as a dictionary, and `generate_structured_with_usage` returns it with an `LLMUsage` record. `StaticLLMProviderRegistry` holds the providers by name. `noop` (returns no candidates) and `rule_based_isnad` (Section 5.8) are always registered; `openai` and `anthropic` are registered only when their API key is set.

- **OpenAI-compatible** (`openai_provider.py`). It posts to `{base_url}/chat/completions` with `temperature` 0 and `response_format` set to `json_object`, and treats an answer cut at the token limit as an error. It reports no token counts.
- **Anthropic** (`anthropic_provider.py`). It calls the Messages API with an `output_config` that carries the effort setting and a JSON schema (`EXTRACTION_WIRE_SCHEMA`). Because the schema allows closed objects only, properties travel as a list of name and value entries and are folded back into a map. The effort comes from `ANTHROPIC_EFFORT` (default `low`). A refusal, a truncated answer and each class of API error become a `StructuredGenerationError` with a `kind`. Usage holds input and output tokens, cache read and cache creation tokens, the stop reason and the model.

The prompt catalog holds two versions, `candidate_graph_extraction_v1` and `candidate_graph_extraction_v2`, which differ in the rule for matching type names; `v2` is the default.

`CachedLLMProvider` wraps a provider and stores one JSON file per call. The key is the SHA-256 hash of the provider name, model, effort, prompt version, prompt text and payload, after the run, schema version, source unit and segment identifiers in the payload are replaced by positional placeholders. A later run with the same

content is therefore a hit, and the stored answer is mapped back to the new run's segment ids. Failed calls are not stored.

The extraction use case calls `generate_structured` and stores no token counts. The cache and the usage records are used by `scripts/run_isnad_experiment.py`, which writes tokens, latency and cost per call and stops issuing calls at a spending limit.

## 6.5 Persistence

**Postgres.** The ORM declares 18 tables. Migration `0001` creates the 15 baseline tables; `0002` adds the node type confidence threshold, `0003` the `processing_jobs` table, `0004` the `pipeline_definitions` table, `0005` the `merge_keys` column of `relation_types`, and `0006` the `review_decisions` table with two indexes. The backend applies the migrations at startup when `AUTO_APPLY_POSTGRES_MIGRATIONS` is true. A job is claimed with `FOR UPDATE SKIP LOCKED` and an advisory lock per run, and a run with a running job gets no second one.

**Neo4j.** `commit_bundle` passes `_write_bundle` to `session.execute_write`, so all statements of a run share one transaction. It merges the source unit and its segments, then each node on (`project_id`, `node_type`, `merge_key`) with the labels `CanonicalNode` and the type name, then each relation with its identity key. Provenance is written as `DERIVED_FROM` to the source unit, `SUPPORTED_BY` to segments, one `RelationEvidence` node per committed relation candidate, and a `GraphDelta` node. Type names are checked against the identifier pattern before they enter a statement. Five indexes are created with `CREATE INDEX ... IF NOT EXISTS` on the first commit of a process: one on the node key and one each on the id property of `SourceUnit`, `Segment`, `RelationEvidence` and `GraphDelta`. No uniqueness constraint is defined. Quotations and offsets stay in the candidate rows in Postgres; the graph links to whole segments.

## 6.6 Dashboard

The dashboard is a Next.js application in Turkish, written for a reader who is not a developer. Its colours and type follow the project's public page. The home page explains the product in two sentences and lists the projects as cards with a status sentence, for example "30 hadis · hepsi grafta · bekleyen yok" (30 hadiths, all in the graph, nothing waiting). Creating a project asks only for a name; the isnad template and its schema are applied automatically.

A project is a sequence of four steps shown as a stepper with counts, and each step is one screen with one main action.

1. **Metin yükle (load text).** The user pastes text, picks a JSON file, or loads ten bundled sample hadiths with one click. The result, duplicates included, is reported in a sentence.
2. **Zincirleri çıkar (extract chains).** The user chooses a method: rules, or a language model if the server has one configured. One button processes every hadith that is waiting and ends with a summary of how many hadiths were written to the graph and how many need a decision.
3. **İncele (review).** Only suggestions that need a person appear, one at a time: the hadith text with the quotation highlighted, the suggested value, one sentence that says why it waits, and three actions (accept, reject, correct). Each review reason code has its own sentence; for the value check it reads "the suggested name does not occur in the quotation the system showed". The reviewer's name is asked once and stored in the browser, and the time between opening an item and deciding it is sent with the decision.
4. **Zincirleri oku (read chains).** The committed hadiths are listed with a search field. For the selected hadith the Arabic text is shown right to left with each narrator mention highlighted at its source offsets. Each chain reads from top to bottom: a narrator, then the link with its transmission wording, its quotation and how it was accepted, written out as "accepted automatically (passed the checks)" or "accepted by a person" with the name and date. Selecting a narrator shows the hadiths the name occurs

in and the names it received from and passed to, with a note that a narrator is identified by the written name.

The schema editor, the step-by-step pipeline runner, the job list and the raw graph explorer are behind one secondary entry, "Gelişmiş" (advanced). A glossary of five terms is reachable from every page. Screenshots of each screen are in `frontend/docs/screens/`.

## 6.7 Testing and continuous integration

`uv run pytest --collect-only -q` collects 285 tests: 127 unit tests in 14 files and 158 integration tests in 21 files. Our run of `uv run pytest -q` gave 273 passed and 12 skipped. Integration tests drive the API through FastAPI's `TestClient` on the in-memory backend with scripted providers, so no test calls a network service.

The 12 skipped tests carry the `real_db` marker. `test_realdb_migrations.py` (4 tests) runs the migration chain on an empty database, compares the result with the ORM models, and checks the downgrade. `test_realdb_isnad_flow.py` (7 tests) runs the isnad flow on Postgres and Neo4j: the same narrator pair in two hadiths gives two edges, a repeated graph write adds nothing, an interrupted write leaves no partial graph, and the lookup indexes exist. `test_realdb_review_decisions.py` (1 test) checks that decisions are stored and appended. These tests run only when `GG_TEST_POSTGRES_DSN`, `GG_TEST_NEO4J_URI` and `GG_TEST_NEO4J_PASSWORD` are set; they create and drop scratch Postgres databases. On 12 October 2026 they were run against PostgreSQL 16 and Neo4j 5.26 in local containers and all 12 passed; on the same day the 214 tests that existed at that point were also run once against those databases and passed. That verification found four defects that the in-memory backend had hidden, among them a graph write that used one transaction per node and could leave a partial graph, and the absence of any Neo4j index. All four are fixed.

The evaluation kit has its own tests in `evaluation/tests/`: 72 tests in six files; 71 pass and one is skipped unless `numpy` and `scipy` are installed, because it compares the scorer with the pilot script. The dashboard has no automated tests.

The CI workflow runs two jobs on every push and pull request. The backend job runs `ruff check src tests`, `mypy src` and `pytest -q`; the frontend job runs `npm ci` and `npm run build`. CI sets no database variables, so it skips the `real_db` tests, and it does not run the evaluation kit's tests.

## 6.8 How to run it

The full stack, from the repository root:

```
cp .env.example .env
docker compose up --build
```

The dashboard is then at `http://localhost:3000` and the API at `http://localhost:8000`.

The backend tests and checks:

```
cd backend
uv sync --extra dev
uv run pytest -q
uv run ruff check src tests
uv run mypy src
```

The isnad demo without databases or an API key, following `docs/guides/isnad-demo.md`:

```
cd backend
REPOSITORY_BACKEND=memory ALLOWED_ORIGINS=http://localhost:3000 \
```

```
  uv run uvicorn app.main:app --app-dir src --port 8000
```

```
cd frontend
NEXT_PUBLIC_API_BASE_URL=http://localhost:8000 npm run dev
```

The guide then creates a project from the `isnad` template, loads hadiths through the bulk endpoint, processes them with the `rule_based_isnad` provider, and reads the chains.

## 6.9 Known limitations

Three capabilities are absent by decision: there is no undo of a committed run, no authentication, and no narrator identity resolution.

The code has these further limits.

- The graph delta row is written to Postgres after the Neo4j transaction. A failure between the two leaves the graph ahead of the metadata.
- Node properties are written with `SET n += $properties`, so a later run overwrites a differing value without a review item.
- Accepting a schema proposal closes the review item and does not change the schema.
- The process endpoint runs inside the request, and the chain endpoints scan all source units and runs of the project.
- The relation detail endpoint has no identity key parameter, so for parallel edges it returns one of them.
- Generating segments again replaces the stored segments with new ids, so the evidence refs of earlier runs point at segments that do not exist.
- The dashboard shows only the first reason when a suggestion has several, computes project counts in the browser from two requests per project, and has no automated tests.
- No licence has been chosen.

# 7 Evaluation

This chapter reports what we measured, on which data, against which reference and with which matching rule. Measured numbers come from files under `giraffegraph-core/evaluation/results/` produced by the scripts described in Chapter 6; statistics of the data sets come from the data manifest [6]. All runs were made on 12 October 2026 on the in-memory backend of the engine. The PostgreSQL and Neo4j backend was verified separately by tests (Section 6.7) and was not used for these runs.

## 7.1 Questions

The evaluation answers five questions.

1. How well does each extraction condition recover the narrator mentions of a hadith?
2. What do the validation checks catch when they are placed between the model and the graph, and what do they cost in lost correct output?
3. Is the model's self-reported confidence a usable signal for deciding what a person should review?
4. How much review work and how much money does each condition generate?
5. How far does a written name identify a narrator?

Questions that we could not answer are listed in Section 7.11.

## 7.2 Data and reference labels

We use two sets of hadiths (Table 7.1). Neither has a human gold standard with two independent annotators, and no source describes its labels as one [6].

**Table 7.1** Evaluation sets.

| Set | Hadiths | Reference mentions | Source of text and labels | Role |
|---|---|---|---|---|
| Pilot | 50, five collections, no Bukhari | 327 | Sanadset 650K [5]; labels produced with regular expressions | Development set. The rule-based segmenter was tuned on it |
| Six books | 300, 50 from each of the six canonical collections | 1,778 | Ifta Sunnah data family [1, 6]; labels are the markup of the Ifta platform as restructured by a third party | Test set. Drawn with a fixed seed after removing every record whose text matches a pilot hadith |

The six-book set was sampled from records between 100 and 3,000 characters with at least two labelled narrators. After normalisation every reference mention occurs in its hadith text, and in all 300 hadiths the mentions appear in text order. A hadith has between 2 and 11 mentions (median 6), and 74 of the 300 contain more than one chain.

The platform labels have known inconsistencies. Narrators inside a compiler's comment or a supporting chain are sometimes labelled and sometimes not, so a system that extracts them is counted wrong in some hadiths. We did not correct the labels.

**How good are the regular-expression labels?** For 253 of the 300 hadiths a matching record exists in Sanadset (text similarity of at least 0.9; 118 are identical after normalisation). Scoring the Sanadset labels

against the Ifta labels gives precision 0.939, recall 0.979 and F1 0.959 (95% interval 0.943 to 0.972) under loose matching, and F1 0.931 under strict matching. The two label sets agree closely on these hadiths. This is agreement between two label sets, not the accuracy of either, and the 253 are the hadiths whose text matched in both corpora, not a random subset.

## 7.3 Protocol

The unit of analysis is the hadith. For each hadith the system's narrator mentions are aligned with the reference mentions by maximum-weight bipartite matching, and matched pairs count as true positives. We report two matching rules, fixed before the runs:

- **Strict:** the normalised strings are equal. Normalisation removes diacritics and punctuation and unifies letter variants.
- **Loose:** the strings are equal, or one contains the other, or their token F1 is at least 0.60.

Precision, recall and F1 are micro-averaged over hadiths. Intervals are 95% percentile bootstrap intervals from 2,000 resamples of hadiths. Differences between two conditions use the same resamples for both. A hadith for which a system returns nothing counts as zero mentions and is never dropped.

Throughout this chapter a "wrong" mention is a proposed mention that is not matched to a reference mention under the loose rule, and a "correct" mention is one that is. Both are statements about agreement with the labels' surface form. Section 7.6 shows what the unmatched mentions consist of.

Strict matching is sensitive to where a mention ends. The loose rule is the one we use for comparisons, and we give strict F1 beside it because the gap between the two is itself informative [31].

## 7.4 Conditions

All conditions run through the same engine: ingestion, segmentation, a provider call, validation, and commit. No review decision is made in any run, so the results show the automatic part of the pipeline only.

**Table 7.2** Extraction conditions.

| Condition | Segmentation | Provider | Schema hints |
|---|---|---|---|
| Rule | Chain span found by the rule-based segmenter, split at the marker ح | `rule_based_isnad`, no model | not used |
| LLM, schema only | Whole hadith, one call | Claude Haiku 5.5 | Type names and JSON contract only; type and property descriptions removed |
| LLM, isnad-aware | Whole hadith, one call | Claude Haiku 5.5 | Descriptions and template hints: copy names verbatim, exclude transmission verbs, the Prophet is not a narrator |
| LLM, isnad-aware, chain span | Chain span from the segmenter, one call per segment | Claude Haiku 5.5 | Same descriptions and hints |

The model runs with effort `low` and structured output. Responses are stored in a content-addressed cache, so each run can be reproduced without new calls.

## 7.5 Extraction results

**Table 7.3** Narrator mention extraction on the six-book test set (300 hadiths, 1,778 reference mentions, Ifta labels). P, R and F1 use loose matching; the interval is for loose F1.

| Condition | P | R | F1 [95% interval] | Strict F1 | Exact chains | TP / FP / FN |
|---|---|---|---|---|---|---|
| Rule | 0.968 | 0.881 | 0.923 [0.910, 0.935] | 0.633 | 0.490 | 1,567 / 51 / 211 |
| LLM, schema only | 0.914 | 0.868 | 0.890 [0.865, 0.914] | 0.826 | 0.567 | 1,543 / 145 / 235 |
| LLM, isnad-aware | 0.997 | 0.919 | 0.956 [0.946, 0.966] | 0.904 | 0.690 | 1,634 / 5 / 144 |
| LLM, isnad-aware, chain span | 0.998 | 0.903 | 0.948 [0.936, 0.959] | 0.897 | 0.650 | 1,605 / 3 / 173 |

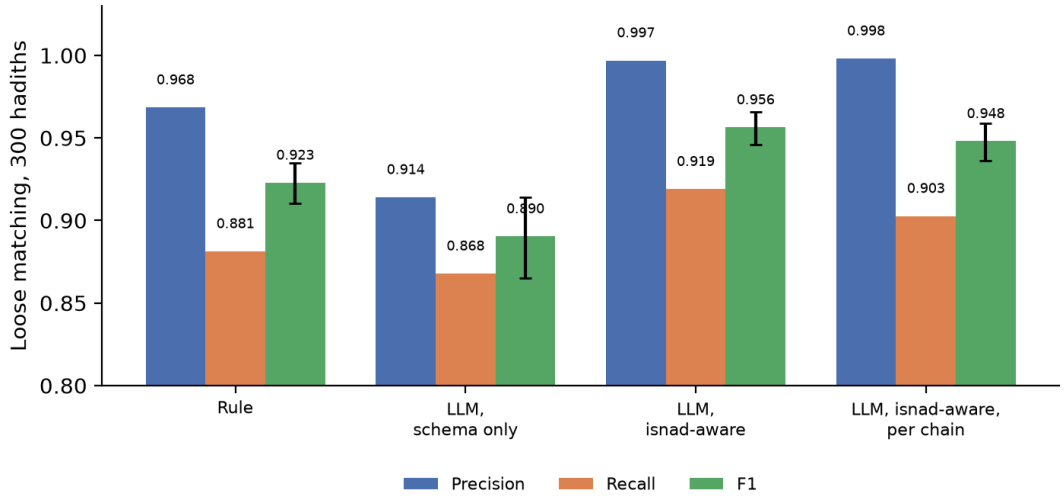



Figure 7.1 Loose precision, recall and F1 by condition on the six-book test set. Error bars on F1 are 95% bootstrap intervals.

"Exact chains" is the share of hadiths in which every mention is matched, in order, with nothing extra, under loose matching.

**Under loose matching a schema alone does not make the model better than rules.** The schema-only model is below the rule-based parser by 0.032 F1 (interval 0.006 to 0.060) and by 0.054 precision (0.026 to 0.083). Recall does not differ (0.013, interval −0.018 to 0.044). Under strict matching the order is reversed: strict F1 is 0.826 for the schema-only model and 0.633 for the parser, and the model also gets more whole chains right (0.567 against 0.490). The parser finds the right names, but its spans often run past the end of a name; the model returns clean names and adds or alters some.

**Descriptions and hints account for most of the gain.** Giving the same model the type and property descriptions and the isnad hints raises precision by 0.083 (0.059 to 0.108), recall by 0.051 (0.024 to 0.078) and F1 by 0.066 (0.041 to 0.091). Unmatched proposals fall from 145 to 5. With hints the model is above the rule-based parser by 0.034 F1 (0.021 to 0.047) and well above it under strict matching. We did not run a condition without a schema, so this is the effect of the instructions given a schema, not a comparison of instructions with a schema.

**Restricting the model to the chain span did not help.** In the fourth condition the model sees only the chain span that the segmenter found. F1 is 0.948 against 0.956 for the whole hadith; the difference of 0.008 has an interval of −0.002 to 0.019. The segmenter starts a new segment only at the chain-switch marker, so only 10 of the 300 hadiths were split into two calls. The condition therefore tests the restriction of the input

far more than it tests one call per chain, and the question whether per-chain calls help on long hadiths with several chains remains open. No call was truncated in either condition. The design analysis of 30 September had recommended per-chain calls after seeing truncated responses with a different model; with Claude Haiku 5.5 and hadiths of this length we do not observe truncation. In the chain-span condition one of 310 calls failed because the engine rejected a relation that referred to an unknown node.

**Table 7.4** Loose recall by structure of the hadith (six-book set).

| Condition | 1 chain (226) | 2+ chains (74) | 2–4 mentions (22) | 5–6 mentions (203) | 7–11 mentions (75) |
|---|---|---|---|---|---|
| Rule | 0.924 | 0.778 | 0.843 | 0.923 | 0.806 |
| LLM, schema only | 0.886 | 0.824 | 0.928 | 0.876 | 0.844 |
| LLM, isnad-aware | 0.940 | 0.868 | 0.976 | 0.948 | 0.854 |
| LLM, isnad-aware, chain span | 0.915 | 0.874 | 0.964 | 0.930 | 0.841 |

All conditions lose recall on hadiths with parallel chains and on long hadiths. The rule-based parser loses the most on parallel chains (0.924 to 0.778). Per collection, loose F1 for the isnad-aware model ranges from 0.936 (Bukhari) to 0.980 (Nasa'i), and for the rule-based parser from 0.862 (Abu Dawud) to 0.963 (Nasa'i).

On the pilot set, scored against the Sanadset labels, the order of the conditions is the same: rule 0.892, schema only 0.828, isnad-aware 0.945, chain span 0.919 (loose F1). The pilot numbers are optimistic for the rule-based parser, which was tuned on those 50 hadiths.

## 7.6 What the checks catch

The checks matter in the condition where the model departs from the text: schema only. Table 7.5 crosses whether a proposed mention is matched to the reference with whether a check flagged it.

**Table 7.5** The value-in-quotation check on the schema-only condition (six-book set, 1,688 proposed mentions, 145 of them not matched to the reference).

| | Not matched | Matched |
|---|---|---|
| Flagged by the check | 122 | 33 |
| Not flagged | 23 | 1,510 |

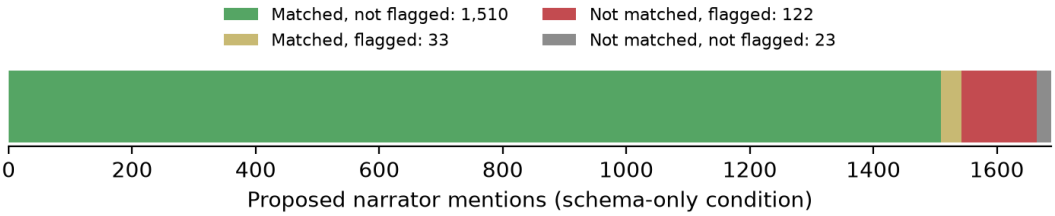



Figure 7.2 Proposed mentions of the schema-only condition by match with the reference and by whether the value-in-quotation check flagged them.

The check flags 155 mentions (9.2% of proposals) in 70 hadiths. Of the flagged mentions 79% are unmatched, and they are 84% of all unmatched mentions. No other node-level check fired: every quotation the model gave was found in its source segment under the comparison the checks use.

What the check detects is that the model wrote a name that is not in the text it quoted. Table 7.6 classifies the 155 flagged names with fixed rules (`evaluation/classify_value_flags.py`).

**Table 7.6** Flagged names of the schema-only condition by kind of change.

| Kind of change | Example (quotation, name) | Flagged | Not matched | Matched |
|---|---|---|---|---|
| Transliterated into Latin script | حَدَّثَنَا الْأَعْمَشُ, "al-A'mash" | 79 | 79 | 0 |
| Grammatical case of the name changed | عَنْ أَبِي بِشْرٍ (genitive), أَبُو بِشْرٍ (nominative) | 34 | 21 | 13 |
| Kinship reference replaced by a name | عَنْ أَبِيهِ ("from his father"), عُرْوَةَ | 25 | 18 | 7 |
| Other, name found elsewhere in the text | name longer than the quotation | 7 | 3 | 4 |
| Other, name not found in the text | حَدَّثَنَا يَحْيَى ، هُوَ ابْنُ سَعِيدٍ, يحيى بن سعيد | 10 | 1 | 9 |
| All | | 155 | 122 | 33 |

Three observations qualify the headline numbers.

First, most of the "wrong" mentions are not invented narrators. Seventy-nine are the right narrator written in Latin script, all from 14 hadiths in which the model switched script. Twenty-one are the right narrator in a different grammatical case. Whether a case change counts as matched depends on the length of the name: on a two-word name the token overlap falls below the threshold of the loose rule, on a longer name it does not. The check and the scorer both compare surface forms, so part of their agreement is by construction.

Second, the kinship cases are inferences, not inventions. In the example the text reads "Hisham ibn Urwa, from his father", and the model wrote "Urwa". The name is in the text; it is not in the quotation. Of the 25 kinship names, 15 occur in the hadith text. We found one flagged name that needs knowledge from outside the text: the quotation is قَالَ عُثْمَانُ and the name is عثمان بن عفان, where the second part occurs nowhere in the hadith.

Third, the classes of Section 2.3 separate as follows. Of the 145 unmatched mentions, 102 have a name that does not occur in the normalised hadith text, and the check flags all 102. Forty-three have a name that does occur in the text; the check flags 20 of them, those whose name is outside the quotation. Ten unmatched mentions name the Prophet as a narrator, a category error; the check flags two of them, both in Latin script, and misses eight, because the name is copied correctly from the text. No implemented check covers this class.

Because a flagged candidate is not accepted until a person decides, the candidates accepted without review in this condition have precision 0.985 instead of 0.914, at recall 0.849 instead of 0.868 (Table 7.7). The graph itself receives less: a run with an open review item is not committed, and 110 of the 300 hadiths have one. Counting those hadiths as empty, the committed output has precision 0.983 and recall 0.582 until a reviewer acts.

**Table 7.7** Output of the schema-only condition at three points (six-book set, loose matching).

| Output | P | R | F1 | TP / FP / FN |
|---|---|---|---|---|
| Everything the model proposed | 0.914 | 0.868 | 0.890 | 1,543 / 145 / 235 |
| Candidates accepted without review | 0.985 | 0.849 | 0.912 | 1,510 / 23 / 268 |

| Output | P | R | F1 | TP / FP / FN |
|---|---|---|---|---|
| Committed to the graph without review (190 hadiths) | 0.983 | 0.582 | 0.731 | 1,034 / 18 / 744 |

In the isnad-aware condition 5 of 1,639 proposals are unmatched, and the check flags 3 of them with no matched mention flagged. The checks have little left to find when the prompt already tells the model to copy names as written. On the pilot set the pattern of the schema-only condition repeats against the Sanadset labels: 35 flagged, all 35 unmatched, 3 unmatched mentions not flagged.

## 7.7 Model confidence

For the schema-only condition, the area under the ROC curve of the model's confidence as a predictor of a matched mention is 0.634. The model used seven distinct confidence values, and 1,623 of the 1,688 mentions carry a value between 0.8 and 1.0.

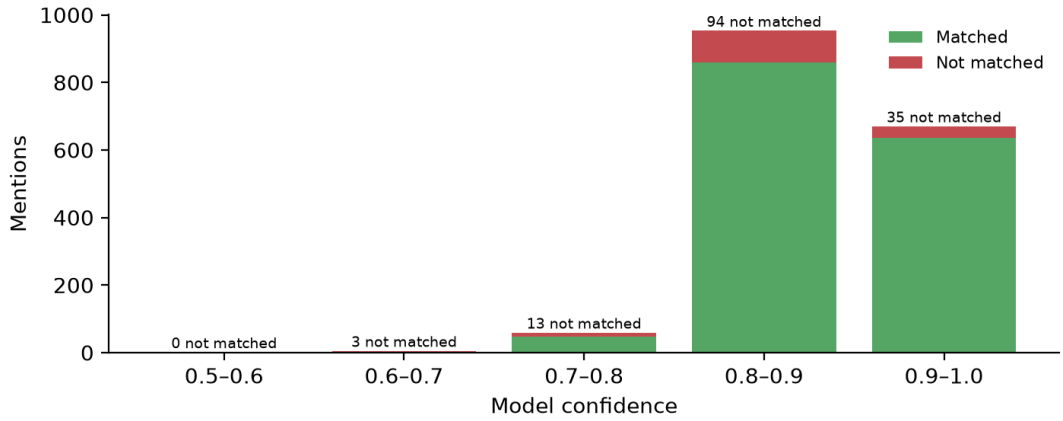



Figure 7.3 Model confidence of matched and unmatched mentions, schema-only condition.

A confidence threshold and a check are comparable only at the same share of mentions sent to review. Table 7.8 lists the three thresholds that select a meaningful share.

**Table 7.8** Unmatched mentions sent to review by a confidence threshold and by the value-in-quotation check (schema-only condition, 145 unmatched mentions in total).

| Signal | Mentions sent to review | Unmatched among them |
|---|---|---|
| Confidence below 0.80 | 65 (3.9%) | 16 |
| Confidence below 0.85 | 275 (16.3%) | 50 |
| Confidence below 0.90 | 1,018 (60.3%) | 110 |
| Value-in-quotation check | 155 (9.2%) | 122 |

At 16% of mentions the confidence threshold finds 50 unmatched mentions; the check finds 122 at 9%. To find 110, the threshold must send 60% of all mentions to a reviewer. The check needs no model call.

## 7.8 Review workload, commits and cost

**Table 7.9** Work left for a reviewer and cost per condition (six-book set).

| Condition | Candidates sent to review | Hadiths with nothing to review | Runs committed without review | Cost per 1,000 hadiths | Median latency per call |
|---|---|---|---|---|---|
| Rule | 0 | 100% | 300 of 300 | 0 | n/a |
| LLM, schema only | 538 | 63% | 190 of 300 | USD 1.87 | 11.1 s |
| LLM, isnad-aware | 16 | 98% | 294 of 300 | USD 2.29 | 13.0 s |
| LLM, isnad-aware, chain span | 6 | 99% | 295 of 300 | USD 2.21 | 12.8 s |

In the schema-only condition the 538 review candidates are 155 nodes flagged by the value check and 383 relations. The relation findings are 192 without a value for an identity property, 208 that depend on a node under review and 9 below the confidence threshold; a relation can carry more than one. A run with an open review item cannot be committed, so 110 of the 300 hadiths wait for a person.

Costs use the price constants in the experiment script, taken from the provider's price list as of 6 October 2026 for Claude Haiku 5.5 (USD 0.10 per million input tokens, USD 0.50 per million output tokens), and the token counts returned by the API: about 3,200 to 3,500 input and 3,100 to 3,900 output tokens per hadith. Latency is the median over calls that were not served from the response cache (300, 300 and 108 calls). The whole evaluation, including the pilot runs and the subset runs in Section 7.9, cost USD 3.49, summed over the eight model runs without counting cached replays.

For every committed fact in every run we checked two properties: its quotation is found in its source segment under the comparison used by the checks, and it has an acceptance record. Both hold for all committed facts. Under byte-exact comparison the first does not hold for the model runs: the model returns Arabic text in a different Unicode composition from the source, and a few quotations differ from the source in vowel marks, which the comparison ignores.

We have no measurement of human review time. The decision record stores it, but no person reviewed these runs.

## 7.9 A second model: a configuration failure

We ran Claude Sonnet 5.5 with the same settings on a stratified subset of 36 hadiths (six per collection) to see whether the results depend on the model. The run did not produce a usable comparison, and we report why.

With effort `low`, Sonnet 5.5 returned a single narrator for 28 of the 36 hadiths in the isnad-aware condition, and in most of those it wrote the narrator's name into the field that holds a property's key. Loose recall was 0.237 (isnad-aware) and 0.530 (schema only) against 0.916 and 0.828 for Haiku 5.5 on the same 36 hadiths.

We suspect two things in our configuration, and did not test either by a further run. The provider sends properties as a list of entries with the fields `name` and `value`, and the property that holds a narrator's name is itself called `name`, which is ambiguous. Second, the effort setting is not calibrated the same way across models. Haiku 5.5 followed the format in all but one of its 910 calls on the six-book set.

The engine handled the malformed output as designed. Unknown property keys were recorded as schema proposals (29 distinct keys in one condition), nodes without their identity property were rejected, 30 of the 36 runs were blocked from commit, and precision of what was proposed stayed at 0.966 or above. Nothing malformed reached the graph. This is the property-level form of schema drift that the type-level check of the original code could not see.

We do not draw a conclusion about the two models from this run. The wire format should be changed before another model is evaluated.

## 7.10 Narrator identity

The engine identifies a narrator node by the written name. The Ifta labels attach a person identifier to each mention, which lets us measure what that choice costs without running any system. The labels list each chain separately, so a narrator shared by parallel chains is counted once per chain. Over the 300 test hadiths this gives 2,150 chain positions filled by 740 people (the 1,778 reference mentions of Table 7.1 count each written mention once).

**Table 7.10** Grouping chain positions by a string key instead of by person (six-book set, reference labels only). Pairwise precision is the share of position pairs with the same key that are the same person, and pairwise recall the share of same-person pairs that share a key. B-cubed averages, over positions, the share of positions with the same key that are the same person (precision) and the share of the person's positions that have the same key (recall).

| Key | Distinct keys | Keys that cover more than one person | Positions under such keys | People split over several keys | Pairwise P / R | B-cubed P / R |
|---|---|---|---|---|---|---|
| Name exactly as written | 964 | 24 (2.5%) | 216 (10.0%) | 191 (25.8%) | 0.737 / 0.559 | 0.941 / 0.715 |
| Normalised name | 878 | 28 (3.2%) | 262 (12.2%) | 161 (21.8%) | 0.759 / 0.703 | 0.931 / 0.792 |

One chain position in ten falls under a name shared by more than one person, and a quarter of the people appear under more than one name. The most ambiguous keys are kinship references: أَبِيهِ ("his father") stands for 34 different people in this sample. Normalising the name reduces splitting and increases conflation. Neither key is an identity. The same pattern is visible in the AR-Sanad test chains [7], where 815 of 15,301 surface names map to more than one identifier.

This is a measurement of the problem, not of a solution. Linking mentions to an authority list is future work.

## 7.11 What we did not measure

- **Human gold standard.** No hadith in this evaluation was labelled by two independent annotators. The annotation sheet and guideline exist; the labelling has not been done.
- **Human review.** No reviewer decided any candidate, so we have no review time, no rejection rate and no measurement of the pipeline with a person in it.
- **Relations.** We scored narrator mentions and their order, not `NARRATED_FROM` links as triples.
- **Other models.** One model produced usable results. The second run failed for a configuration reason.
- **Schema drift.** The Haiku runs produced one unknown property in 910 calls and no unknown type, so the measured evidence for drift comes almost entirely from the failed second-model run.
- **Run-to-run variation.** Each condition was run once.
- **Memorisation.** We did not test whether the model recalls well-known chains instead of reading them.
- **Question answering.** The graph is not yet compared with text retrieval on any question set.
- **Text integrity.** We did not compare the corpus text with a printed edition.

## 7.12 Earlier pilot

A pilot on 30 September 2026 used the same 50 hadiths with a different model (Claude Sonnet 5) outside the engine. It reported loose F1 of 0.879 for a rule-based parser, 0.865 for a schema-only prompt and 0.935 for an isnad-aware prompt; 605 of 605 model quotations found in the source; 32 of 318 names not found in the

model's own quotation; and a confidence AUC of 0.626. The engine-based runs in this chapter reproduce each of these patterns with a different model, on six times as much unseen data and against a different label source.

# 8 Discussion

## 8.1 Where the check belongs

The project began with the claim that tying each fact to a source quotation keeps unsupported facts out of a graph. In our runs the quotation was not where the model departed from the text. Every quotation that reached the check was found in its segment. What differed was the relation between the quotation and the extracted value: the model quoted "from his father" and wrote a name, or quoted a name in Arabic and wrote it in Latin letters.

A quotation check answers "did the model point at real text". The value-in-quotation check answers "is what the model wrote in the text it pointed at". Only the second question separated matched from unmatched proposals in our data. Both checks cost a string comparison.

The check also flags values a reviewer would accept. Of the 155 flagged names, 33 match the reference (21%), and 13 of those differ from the quotation only in grammatical case. This is why the engine sends a flagged candidate to review instead of rejecting it. A rule that recognises case endings as an allowed change would accept those 13 automatically and record that it did so; it would not remove the other 20.

The check does not tell an invented narrator from a transformed one. Most flagged names are the right person in another script or case, and most kinship cases are inferences the text supports. What the check establishes is narrower and still useful: the model changed something and did not say so. For a graph whose facts must be traceable to the wording of a source, that is the event that needs a decision.

## 8.2 What the model is needed for

Under loose matching a rule-based parser with no model call reaches F1 0.923, a model with only a schema 0.890, and a model with descriptions and hints 0.956.

The comparison between parser and schema-only model depends on the matching rule. The parser finds the names but returns spans that run long, so it does well under loose matching and badly under strict matching (0.633). The schema-only model returns clean names, so it does better under strict matching (0.826) and gets more whole chains right, but it adds and alters names. Which is preferable depends on whether a later step can trim spans or must correct names.

A schema is not domain knowledge. The schema-only model knew the types and still treated the Prophet as a narrator and transliterated names. A few sentences of description and instruction in the schema template removed almost all unmatched proposals. In this task the instructions changed the result more than any check did. We did not test instructions without a schema.

The model with hints is above the parser by 0.034 F1 under loose matching, mostly through recall on hadiths with parallel chains, and by much more under strict matching. Whether that is worth a model call per hadith depends on the use. For a first pass over a large corpus the parser costs nothing. The engine supports both through the same pipeline.

When the prompt is good the checks find little. They are what protects the graph when the prompt, the model or the configuration is not good. The failed run with a second model is an example: the output was malformed in most calls, and nothing malformed was committed.

## 8.3 Confidence

With this model and prompt, the model's confidence was a poor basis for routing. At a comparable share of mentions sent to review it found fewer than half as many unmatched mentions as the check (Table 7.8). The original code sent candidates to review by a confidence threshold alone. The current code keeps the threshold and adds the checks.

## 8.4 The cost of a strict commit rule

The checks keep altered names out of the accepted output, and the commit rule keeps a whole hadith out of the graph while any of its candidates waits for review. In the schema-only condition that leaves 110 of 300 hadiths uncommitted and the graph at recall 0.582 until a person acts. With a good prompt the cost almost disappears (6 of 300). The design moves work to the reviewer; it does not remove it, and we have not measured how long that work takes.

## 8.5 Threats to validity

**Reference labels.** The Ifta labels are platform markup restructured by a third party, with inconsistencies we observed and did not repair. A system is sometimes penalised for extracting a narrator the platform did not mark. Two label sets produced in different ways agree closely on the hadiths they share (F1 0.959), which suggests the labels are adequate for comparing conditions. We do not know how independent the two sets are, and absolute values should not be read as accuracy.

**Matching rule.** Loose matching accepts a mention that contains extra words, and it treats the same change of case as a match or a mismatch depending on the length of the name. Conclusions that hold under one rule and not the other are stated with the rule.

**What "wrong" means.** An unmatched proposal is not necessarily a false statement about the hadith. More than half of the unmatched proposals of the schema-only model name the right narrator in another script.

**One model, one effort setting, one run.** All usable results come from Claude Haiku 5.5 at low effort, with each condition run once. The second model failed for a reason we attribute to our own wire format and effort setting without having tested that attribution.

**In-memory backend.** The experiments ran on the in-memory backend. The same pipeline is covered on PostgreSQL and Neo4j by twelve tests, not by the experiments.

**Tuning.** The rule-based segmenter was tuned on the 50 pilot hadiths. The six-book set excludes them, so the main results are not affected, but the parser's rules reflect the collections seen in the pilot, which did not include Bukhari.

**Memorisation.** The hadiths are well known and are likely to be in the model's training data. We found one flagged name that the text does not contain in any form, which is too little to say whether the model recalls chains. We did not test this with altered names.

**No human in the measured loop.** The review queue, the decision record and the commit guards are covered by tests, but the evaluation ran without a reviewer. The claim that the engine reduces errors with a person in the loop is not measured here.

**Schema drift.** The title names schema drift. With Haiku 5.5 we observed almost none, and what we observed with the second model came from a failed configuration. The engine's handling of drift is tested; its frequency in normal use is not established.

**Authorship of the code and analysis.** Most of the code on the evaluated branch, the analysis scripts and a first draft of this text were produced with AI coding assistants under the authors' direction. The measurements are reproducible from the cached model responses and the scripts. The authors remain responsible for every statement.

## 8.6 Limits of the system

A narrator node is identified by its written name. Section 7.10 shows the size of the resulting error in the reference labels: a graph keyed this way merges different people who share a short name and splits one person across spellings. Any analysis of the narrator network built on the current graph inherits this.

Chains that begin after the report text has started are not segmented, and joined co-narrators stay in one mention. Both reduce the recall of the conditions that use the segmenter.

No check covers category errors such as the Prophet named as a narrator. The checks apply to candidates that come from an extraction call, not to candidates entered by hand. Acceptance records are computed from stored decisions and validation results when requested. Undo of a committed run is not implemented.

# 9 Conclusion and future work

We built an engine that places deterministic checks and a recorded acceptance between a language model and a knowledge graph, and we measured what that placement achieves on hadith narrator chains with one model and platform-derived labels.

The measurements support four statements within those limits. A check on the relation between the model's quotation and the value it extracted flags most proposals of a weakly instructed model that do not match the reference, with no extra model call, while a check on the quotation alone flags none. Adding descriptions and domain instructions to a schema-only prompt improves extraction more than any check does. The model's confidence finds far fewer of those proposals than the check at a comparable review budget. And a written name is not a narrator's identity: one chain position in ten is ambiguous at the level of the string.

The measurements do not show how the engine performs with a reviewer, how the results change with other models, how often schema drift occurs, or how accurate the output is against labels produced by independent human annotators.

## 9.1 Plan for Senior Design Project II

1. **Human-labelled reference.** Two annotators label the 50 pilot hadiths with the existing sheet and guideline, we measure agreement and time per hadith, revise the guideline, and then label a sample of the six-book set. The number of hadiths follows from the measured time.
2. **Reviewer in the loop.** A reviewer who did not label a hadith decides its review items in the dashboard. We report precision and recall after review, rejection rate per reason code and seconds per decision from the decision record.
3. **Narrator identity.** Link mentions to the narrator catalogue of the Ifta data, present ranked candidates to the reviewer for ambiguous names, and report wrong merges separately from missed merges.
4. **Transformation rules and a category rule.** Accept changes of grammatical case automatically and record the rule that accepted them, which would clear 13 of the 33 matched names the value check flagged. Add a rule for names that are not narrators, such as the Prophet.
5. **Models and wire format.** Rename the fields of the property list, then repeat the evaluation with at least two more models, including an open one, with repeated runs and with altered names to test memorisation.
6. **Relations and chains.** Score `NARRATED_FROM` links and whole chains, segment chains that start after the report text, and test per-chain calls on hadiths with several chains.
7. **Questions over the graph.** Build a small set of structural, path and decision questions and compare answers from the graph with answers from text retrieval.

## 9.2 Beyond hadith

The checks, the review queue, the decision record and the relation identity work on any schema, and most hadith behaviour lives in a segmentation strategy, a template and a provider. Three places in the engine still name the case study (Section 5.1). The claim that the same structure serves another domain is untested, and a second case study is the way to test it.

# References


[1] J. M. Oumer, "Relation extraction for graph-based representation of Hadith Isnad," M.S. thesis, University of Malta, Faculty of Information and Communication Technology, Department of Artificial Intelligence, 2025. https://www.um.edu.mt/library/oar/handle/123456789/137629

[2] A. B. Kamran, B. Abro, and A. Basharat, "SemanticHadith: An ontology-driven knowledge graph for the hadith corpus," *Journal of Web Semantics*, vol. 78, pp. 100797, 2023. doi:10.1016/j.websem.2023.100797

[3] A. B. Kamran, N. A. Butt, and A. Basharat, "Semantic Enrichment of Hadith Corpus—Knowledge Graph Generation From Islamic Text," *Semantic Web*, vol. 17, no. 2, pp. 1–27, 2026. doi:10.1177/22104968261431425

[4] T. Wirahman, "Linking Hadith Narrator Identities Across Heterogeneous Arabic Biographical Databases: A Multi-Signal Entity Resolution Pipeline," arXiv:2607.05424, 2026. https://arxiv.org/abs/2607.05424

[5] M. Mghari, O. Bouras, and A. El Hibaoui, "Sanadset 650K: Data on Hadith narrators," *Data in Brief*, vol. 44, pp. 108540, 2022. doi:10.1016/j.dib.2022.108540

[6] GiraffeGraph project, data manifest: sources, licence statements and construction of the evaluation sets, `data/README.md`, 12 October 2026 (unpublished).

[7] S. Mahmoud, O. Saif, E. Nabil, M. Abdeen, M. ElNainay, and M. Torki, "AR-Sanad 280K: A Novel 280K Artificial Sanads Dataset for Hadith Narrator Disambiguation," *Information*, vol. 13, no. 2, pp. 55, 2022. doi:10.3390/info13020055

[8] M. Aslan, "Tabakat Kitapları ve Hadis İlmine Sağladıkları Faydalar," *Bingöl Üniversitesi İlahiyat Fakültesi Dergisi*, vol. 5, no. 10, pp. 269–290, 2017.

[9] M. A. Haque and R. Islam, "Hadith computational science in the age of large language models: a critical narrative review," arXiv:2608.20364, 2026. https://arxiv.org/abs/2608.20364

[10] A. Hogan et al., "Knowledge Graphs," *ACM Computing Surveys*, vol. 54, no. 4, pp. 1–37, 2021. doi:10.1145/3447772

[11] M. Kejriwal, *Domain-Specific Knowledge Graph Construction*. Springer International Publishing, 2019. doi:10.1007/978-3-030-12375-8

[12] H. Ye, N. Zhang, H. Chen, and H. Chen, "Generative Knowledge Graph Construction: A Review," in *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pp. 1–17, 2022. doi:10.18653/v1/2022.emnlp-main.1

[13] S. Pan, L. Luo, Y. Wang, C. Chen, J. Wang, and X. Wu, "Unifying Large Language Models and Knowledge Graphs: A Roadmap," *IEEE Transactions on Knowledge and Data Engineering*, vol. 36, no. 7, pp. 3580–3599, 2024. doi:10.1109/tkde.2024.3352100

[14] B. Zhang and H. Soh, "Extract, Define, Canonicalize: An LLM-based Framework for Knowledge Graph Construction," in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pp. 9820–9836, 2024. doi:10.18653/v1/2024.emnlp-main.548

[15] Z. Ji et al., "Survey of Hallucination in Natural Language Generation," *ACM Computing Surveys*, vol. 55, no. 12, pp. 1–38, 2023. doi:10.1145/3571730

[16] N. Mihindukulasooriya, S. Tiwari, C. F. Enguix, and K. Lata, "Text2KGBench: A Benchmark for Ontology-Driven Knowledge Graph Generation from Text," arXiv:2308.02357, 2023. doi:10.48550/arXiv.2308.02357

[17] S. Min et al., "FActScore: Fine-grained Atomic Evaluation of Factual Precision in Long Form Text Generation," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pp. 12076–12100, 2023. doi:10.18653/v1/2023.emnlp-main.741

[18] J. Wei et al., "Long-form factuality in large language models," in *Advances in Neural Information Processing Systems 37*, pp. 80756–80827, 2024. doi:10.52202/079017-2567

[19] H. Rashkin et al., "Measuring Attribution in Natural Language Generation Models," *Computational Linguistics*, pp. 1–66, 2023. doi:10.1162/coli_a_00490

[20] P. Laban, T. Schnabel, P. N. Bennett, and M. A. Hearst, "SummaC: Re-Visiting NLI-based Models for Inconsistency Detection in Summarization," *Transactions of the Association for Computational Linguistics*, vol. 10, pp. 163–177, 2022. doi:10.1162/tacl_a_00453

[21] Y. Zha, Y. Yang, R. Li, and Z. Hu, "AlignScore: Evaluating Factual Consistency with A Unified Alignment Function," in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 11328–11348, 2023. doi:10.18653/v1/2023.acl-long.634

[22] L. Gao et al., "RARR: Researching and Revising What Language Models Say, Using Language Models," in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 16477–16508, 2023. doi:10.18653/v1/2023.acl-long.910

[23] S. Dhuliawala et al., "Chain-of-Verification Reduces Hallucination in Large Language Models," in *Findings of the Association for Computational Linguistics ACL 2024*, pp. 3563–3578, 2024. doi:10.18653/v1/2024.findings-acl.212

[24] S. Kadavath et al., "Language Models (Mostly) Know What They Know," arXiv:2207.05221, 2022. https://arxiv.org/abs/2207.05221

[25] M. Xiong et al., "Can LLMs Express Their Uncertainty? An Empirical Evaluation of Confidence Elicitation in LLMs," arXiv:2306.13063, 2023. https://arxiv.org/abs/2306.13063

[26] Y. E. Kuruçay, "The Cache and the Ledger," project essay, GiraffeGraph workspace, `cache-and-ledger/README.md`, 2026 (unpublished).

[27] A. N. Angelopoulos and S. Bates, "A Gentle Introduction to Conformal Prediction and Distribution-Free Uncertainty Quantification," arXiv:2107.07511, 2021. https://arxiv.org/abs/2107.07511

[28] A. Z. Raja, "Grading the Narrators: An Isnad-Rijal Framework for Claim-Level Provenance in Multi-Agent Knowledge Systems," arXiv:2607.24117, 2026. https://arxiv.org/abs/2607.24117

[29] "Sanadset 650K (Hugging Face mirror, CC0-1.0)," 2022.

[30] S. Altammami, E. Atwell, and A. Alsalka, "Text Segmentation Using N-grams to Annotate Hadith Corpus," in *Proceedings of the 3rd Workshop on Arabic Corpus Linguistics*, pp. 31–39, 2019. https://aclanthology.org/W19-5605/

[31] R. Muther and D. Smith, "Tracing Traditions: Automatic Extraction of Isnads from Classical Arabic Texts," in *Proceedings of the Fifth Arabic Natural Language Processing Workshop*, pp. 130–138, 2020. https://aclanthology.org/2020.wanlp-1.12/

[32] M. Asgari-Bidhendi, M. A. Ghaseminia, A. Shahbazi, S. A. Hossayni, N. Torabian, and B. Minaei-Bidgoli, "Rezwan: Leveraging Large Language Models for Comprehensive Hadith Text Processing: A 1.2M Corpus Development," arXiv:2510.03781, 2025. https://arxiv.org/abs/2510.03781

[33] O. Çinici, "Hadis Araştırmalarında Büyük Dil Modelleri (LLM): Teorik Çerçeve, Sınırlar ve Yapay Zekâ Destekli Yapılandırılmış Veri İnşası," *Mevzu: Sosyal Bilimler Dergisi*, no. 16, pp. 299–326, 2026. doi:10.56720/msbd.2026.s16.326

[34] B. Mo et al., "KGGen: Extracting Knowledge Graphs from Plain Text with Language Models," in *Advances in Neural Information Processing Systems 38*, pp. 33962–33985, 2025. doi:10.52202/085713-1010

[35] Y. Lairgi, L. Moncla, R. Cazabet, K. Benabdeslem, and P. Cléau, "iText2KG: Incremental Knowledge Graphs Construction Using Large Language Models," in *Web Information Systems Engineering – WISE 2024*, pp. 214–229, 2024. doi:10.1007/978-981-96-0573-6_16

[36] J. Gao, X. Li, Y. E. Xu, B. Sisman, X. L. Dong, and J. Yang, "Efficient knowledge graph accuracy evaluation," *Proceedings of the VLDB Endowment*, vol. 12, no. 11, pp. 1679–1691, 2019. doi:10.14778/3342263.3342642

[37] A. Mhamedi, M. Mghari, and A. El Hibaoui, "Computational methodologies for sanad-based hadith analysis: a review," *TELKOMNIKA (Telecommunication Computing Electronics and Control)*, vol. 24, no. 3, pp. 840–851, 2026. doi:10.12928/telkomnika.v24i3.27447

# Appendix A Reproducing the results

All commands are run from `giraffegraph-core/`. The evaluation data is described in `data/README.md` in the project workspace and is not redistributed.

Backend tests, with no database or network:

```
cd backend
uv sync --extra dev
uv run pytest -q
```

Tests against real databases need a PostgreSQL 16 and a Neo4j 5 instance and three environment variables:

```
GG_TEST_POSTGRES_DSN=... GG_TEST_NEO4J_URI=... GG_TEST_NEO4J_PASSWORD=... uv run pytest -q -m real_db
```

One evaluation run and its analysis:

```
cd backend
uv run python scripts/run_isnad_experiment.py --input <hadiths.json> --out-dir <run-dir> \
    --condition llm_isnad_whole --model claude-haiku-5-5 --effort low --max-cost-usd 0.85
cd ..
python evaluation/analyze_experiment.py --run <run-dir> --reference <reference.jsonl> --out <run-
dir>/analysis
python evaluation/compare_runs.py --runs <run-dir> <run-dir> ... --reference <reference.jsonl> --out
compare.md
python evaluation/extra_tables.py --runs <run-dir> ... --reference <reference.jsonl> \
    --reference-chains <reference_chains.jsonl> --out extra-tables.json
python evaluation/classify_value_flags.py --run <run-dir> --hadiths <hadiths.json> \
    --reference <reference.jsonl> --out value-flag-classes.json
python evaluation/identity_baseline.py --reference-chains <reference_chains.jsonl> --out
identity.json
```

Model responses are cached under `backend/var/llm-cache/`. With the cache present a run makes no API call and reproduces the same output.

# Appendix B Result files

| Content | Path under `giraffegraph-core/evaluation/results/` |
|---|---|
| Six-book test set, four conditions | `2026-10-12-sixbooks300/` (`compare-ifta.md`, `extra-tables-ifta.json`, one folder per run; `haiku-llm_schema_whole/value-flag-classes.json`) |
| Pilot set, four conditions | `2026-10-12-pilot50/` (`compare-sanadset.md`) |
| Second-model subset | `2026-10-12-subset36/` (`compare-ifta.md`) |
| Narrator identity baseline | `2026-10-12-identity-baseline/sixbooks300.json` |
| Rule-based baseline of the first end-to-end run | `2026-10-12-rule-based-baseline/` |

Each run folder holds the system output in the scorer's format, one line per candidate with its status and reason codes, one line per provider call with tokens and latency, a run report and the analysis.

# Appendix C Use of AI tools

AI assistants were used throughout this project: for the literature search, for the design analysis and pilot of 30 September 2026, for most of the code written on the evaluated branch, for the analysis scripts, and for drafting this report. The extraction experiments themselves use a language model as the system under test.

To limit the risks of this way of working we applied the following rules. Bibliographic records were retrieved from Crossref, arXiv, the ACL Anthology or the publisher's first page, never written from memory; three references in an earlier project document could not be found in any index and are not used here. Every measured number in this report was produced by a script and is stored in a result file; statistics of the data sets are taken from the data manifest. Code was accepted only with tests, and the tests marked for real databases were run against PostgreSQL and Neo4j. A draft of this report was checked by an independent AI reviewer against the result files, and the report was corrected where it claimed more than the files show. Statements about other work are limited to what was read in the cited source.

The authors are responsible for the content of this report.